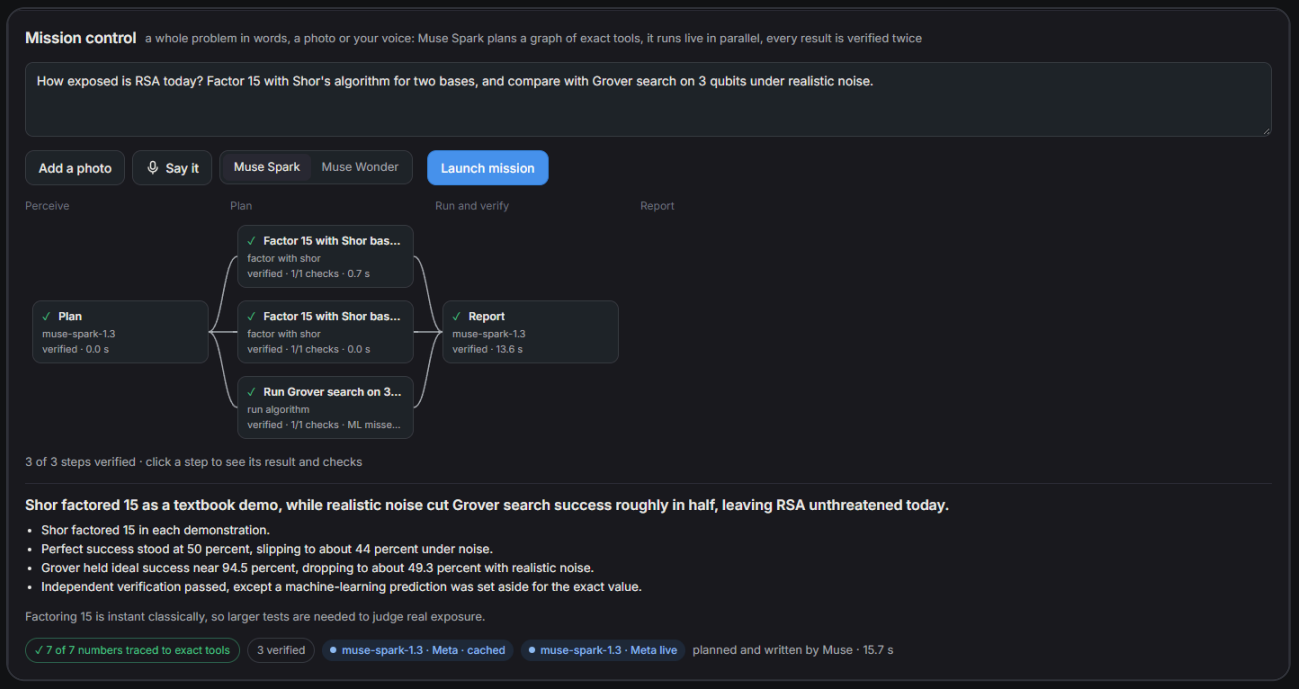
META GLOBAL AI DEVELOPER HACKATHON 2026 · PUSH THE FRONTIER · INDIE TRACK

Inkubit

Meta's models propose.
Exact physics checks every number.

Technical report How it works, what it found, and how every claim is checked



Muse Spark 1.3 · Muse Wonder 1.0 · SAM 3.1 · Muse Image 1.0 · Muse Voice Transcribe 1.0 · Qiskit + Aer

---

0.814 vs 0.795

GHZ fidelity on IBM's ibm_fez: Muse Spark's 7-qubit circuit against the textbook's

68 of 68

steps verified in a 30-mission live benchmark planned by Muse Spark

591 + 6

backend and browser tests, run in CI on every push

About this report

This report describes Inkubit: what it is for, how it is built, how it uses Meta's models, what its labs found, and how each result was checked. It is for hackathon judges, and for anyone who wants to know how the project works beyond the demo video.

Inkubit started from one observation. A language model can describe a quantum experiment fluently, but it can't tell you whether its own numbers are right. Inkubit makes the models do what they are good at: reading sketches, photos, papers and speech, planning multi-step work and proposing circuits and codes. Everything that becomes a number is computed by exact physics: density-matrix simulation, an independent second simulator, exhaustive checkers and physical invariants. Every number a model writes in an answer is traced back to the tool that computed it.

The report holds itself to the same rule. Each number in a chart or a generated table, and each benchmark result quoted in the prose, is read from the repository's data files when the report is built, by `scripts/build_report.py`. These are the same files the app shows and the test suite checks. A few numbers come from features that keep no stored result, such as live runs of the network planner or the discovery lab. Those are quoted from the README and the decision log, which record the runs that produced them, and the text says so where it matters. The test `tests/test_claims.py` already fails the build if the README or the submission text quotes a number its data no longer supports. This report is generated from the same data, so it cannot drift from it either.

Project	Inkubit, a quantum lab where the AI never gets the last word
Event	Meta Global AI Developer Hackathon 2026; building period 27 October to 4 November 2026
Track and problem	Indie developer; Push the Frontier (accelerate a scientific experiment; understand and control a physical system)
Built by	One developer, during the building period; first commit 27 October 2026 at 14:57 ET
Code	github.com/Dhruv-Deku/inkubit, MIT licence
Meta models	Muse Spark 1.3, Muse Wonder 1.0, SAM 3.1, Muse Image 1.0, Muse Voice Transcribe 1.0
This edition	Built 3 November 2026 from commit f0beca5 (100 commits)

How the report is organised

Chapters 1 and 2 give the summary and the idea. Chapters 3 and 4 cover the architecture and how each Meta model is used. Chapters 5 and 6 cover the two main ways in: turning a sketch into verified code, and mission control, which turns a whole problem into a plan of exact tools. Chapters 7 to 9 cover the research labs where the AI proposes and physics decides: discovery, learning under proof, AI-designed error-correcting codes and reproducing the literature. Chapters 10 to 12 go down to the hardware (decoders, pulses, distributed machines, resource estimates) and out to orbit. Chapters 13 and 14 cover the tools for looking inside a circuit, and the labs and games. Chapter 15 collects the evidence, including the run on a real IBM quantum computer. Chapters 16 to 18 cover engineering, limitations and the build timeline. The appendices list the API, the decision index, the commands that reproduce every result, a glossary and references.

Contents

1 **Summary** 4
- 1.1 What Inkubit is 4
- 1.2 Results at a glance 4
- 1.3 By the numbers 5
- 1.4 What makes it different 5

2 **The problem and the principle** 6
- 2.1 A wrong quantum program still runs 6
- 2.2 Language models are fluent, not exact 6
- 2.3 The principle: the AI proposes, exact physics has the last word 6
- 2.4 Rules the app keeps 7
- 2.5 Who it is for 7

3 **Architecture** 8
- 3.1 Overview 8
- 3.2 The backend 8
- 3.3 The frontend 9
- 3.4 The model layer 9
- 3.5 Jobs, events and streaming 10
- 3.6 Technology 10

4 **Meta's models in Inkubit** 11
- 4.1 Five models, each where it is strongest 11
- 4.2 Structured output as the contract 11
- 4.3 Tool calling and number tracing 12
- 4.4 Seeing: SAM 3.1 and set-of-marks prompting 12
- 4.5 Voice and images 12
- 4.6 Reliability and cost 13
- 4.7 An open-weight model on a laptop 13

5 **Sketch to verified code** 14
- 5.1 Reading a sketch 14
- 5.2 The specification 15
- 5.3 Writing and running code safely 15
- 5.4 Verification 15
- 5.5 Diagnosis and repair 16
- 5.6 The planted-fault benchmark 16
- 5.7 Understanding the result 17

6 **Mission control** 18
- 6.1 From one request to a plan 18
- 6.2 Running in parallel 18
- 6.3 Verification that does not trust the tools 19
- 6.4 Reports with every number traced 19
- 6.5 The 30-mission benchmark 19

7 **Discovery: AI proposes, physics proves** 22
- 7.1 The discovery lab 22
- 7.2 An AI scientist that learns under proof 23
- 7.3 What the curves show 24

8 **AI-designed error-correcting codes** 25
- 8.1 Stabilizer codes in brief 25
- 8.2 An exact checker 25
- 8.3 What Muse Spark designed 25
- 8.4 Reading the result honestly 26

9 **Reading the literature** 27
- 9.1 Reproduce a figure 27
- 9.2 Reproduce a whole paper 27
- 9.3 What "not reproduced" means 28
- 9.4 A silent error that a test caught 28

10 **Error correction and learned decoders** 29
- 10.1 The error-correction lab 29
- 10.2 The surface code and its decoders 29
- 10.3 Distance 3: a network beats matching 30

10.4 Distance 5: a bigger code, fewer errors 30
10.5 A network on top of matching 30

11 **Down to the hardware** 32
11.1 The pulse lab 32
11.2 A distributed quantum computer 33
11.3 When does quantum win? 33

12 **Quantum networks and satellites** 34
12.1 The link model 34
12.2 Bottleneck search and purification 34
12.3 Satellites versus fiber 35
12.4 Real satellites, live 35

13 **Inside a circuit** 36
13.1 Gate by gate 36
13.2 Shorter circuits, proven equal 36
13.3 Zero-noise extrapolation 37
13.4 Bell tests and tomography 37
13.5 The Quantum Rosetta 38

14 **Labs and games** 39
14.1 Challenges 39
14.2 Bug hunt 39
14.3 Split the party (QAOA) 39
14.4 H_2 from first principles 40
14.5 Catch the spy (BB84) 40
14.6 The algorithm library 40

15 **Evidence** 41
15.1 AI alone vs Inkubit 41
15.2 The AI referee 42
15.3 The ML verifier 42
15.4 A real IBM quantum computer 43
15.5 AI-discovered circuits on the chip 45
15.6 IBM Quantum: placement and teleportation 46
15.7 The proof sheet: every check on every circuit 47
15.8 The simulation observatory 48

16 **Engineering** 49
16.1 Tests 49
16.2 Our own numbers, held to the app's rule 49
16.3 Continuous integration and browser tests 50
16.4 Security and public mode 50
16.5 Hosting on a free container 50
16.6 The demo video, recorded by a script 51

17 **Limitations and next steps** 51
17.1 Limitations 51
17.2 Next steps 51

18 **Built during the hackathon** 52
18.1 The record 52
18.2 Day by day 53
18.3 How the work was done 53

A **API reference** 54

B **Decision index** 55

C **Reproducing every result** 56
C.1 Set up and test 56
C.2 Results and the scripts that produce them 57

D **Glossary** 57

E **References** 59

CHAPTER 1

Summary

Inkubit is a quantum lab in the browser. Meta's models read, plan and propose, and exact physics checks every number before anyone sees it, from a Bell pair drawn in pen to a distance-5 surface code, and from satellites in orbit tonight to a real IBM quantum computer.

1.1 What Inkubit is

You can draw a circuit on paper, photograph a whiteboard, type a problem, say it out loud or give it a paper. Inkubit turns any of these into a structured plan. SAM 3.1 finds the drawn parts of a sketch. Muse Spark reads the photo into a validated specification, writes and repairs code, plans multi-step missions and writes reports. Muse Wonder takes the long, many-step questions and reads whole papers. Muse Voice Transcribe takes speech, and Muse Image draws illustrations that are always shown next to the exact drawing.

None of these models produces a physical result itself. The results come from exact engines: density-matrix simulation with realistic noise, an independent second simulator, exhaustive search, symplectic checkers for error-correcting codes, an SGP4 orbit propagator, closed-form chemistry integrals and a minimum-weight matching decoder. These run under physical invariants and statistical tests. If a check fails, Inkubit says so: a run that cannot be repaired is shown as **Not verified**, and a number in a model's answer that matches no tool result is flagged.

The same principle runs through every lab. A model proposes a circuit layout, an error-correcting code, a plan or a reading of a paper. An exact method then accepts or rejects the proposal and says why. In some labs the verdict goes back to the model, which then does better: Chapter 7 shows a model that improves only when it sees exact physics' verdicts.

1.2 Results at a glance

WHERE	MEASURED RESULT
A real IBM quantum computer	The discovery lab's GHZ circuit for 7 qubits ran on IBM's ibm_fez next to the textbook circuit, on the same seven qubits: GHZ fidelity 0.814 against 0.795 (3.2 standard errors), both certifying genuine 7-qubit entanglement. A noise preset calibrated from the chip predicted four earlier circuits within 0.013 to 0.027, closer than a digital twin (0.021 to 0.038).
IBM Quantum	The same GHZ-5 circuit on the advisor's best, median and worst qubits of ibm_fez: 0.894, 0.814 and 0.285, in its order. Teleportation with real mid-circuit feed-forward: 0.924 against a classical limit of 0.667.
Mission control	In a 30-mission live benchmark, 30 of 30 finished, 68 of 68 steps were verified (1 repaired by the model) and 204 of 209 report numbers were traced to a tool result. Median time per mission: 15 s.
An AI scientist that learns	With exact feedback, Muse Spark's best proven circuit improved on every target (6 to 5, 6 to 5 and 8 to 7 CNOTs). Asked afresh each round, it never improved on its first answer. Random wiring never reached a target.
AI-designed codes	For noise dominated by phase flips, Muse Spark found a 9-qubit code that needs 9 Z errors to fail, against the surface code's 3, while still correcting any single error.

WHERE	MEASURED RESULT
Discovery lab	A GHZ state on all 7 qubits of an IBM heavy-hex chip in 6 CNOTs (the textbook circuit compiled onto the chip needs 10). W and Dicke states in 5 CNOTs, where Qiskit's compiler uses 11.
Learned decoders	At distance 3, a network makes 21% fewer logical errors than minimum-weight matching at 1% noise. At distance 5, the bigger code suppresses errors 5.9× per round, and a network on top of matching makes 10% fewer errors again (paired z = 7.8).
Reproducing a whole paper	Muse Wonder found 8 experiments in a 100-page paper (arXiv:1804.03719). Exact simulation reproduced 3 of the 5 it could check (11 of 19 stated numbers) in 107 s.
The AI referee	Muse Wonder challenged Muse Spark's answers. The debate raised them from 17 to 19 of 24 right, but Wonder also raised 5 false alarms. Only exact simulation knew which challenge was right.
AI alone	The same model, with no simulator, was exact on 13 of 13 textbook circuits but on only 3 of 6 bigger new ones, once 43% off with full confidence.
ML verifier	Predicts how much of a circuit's answer survives noise in about 151 µs, 9× more accurately than the usual rule of thumb, with a calibrated error bar. Each prediction is checked exactly.
When does quantum win?	The surface-code resource model lands within 1.3× (qubits) and 2.3× (time) of Gidney and Ekerå's published RSA-2048 estimate, and it says "classical wins" when classical wins.

1.3 By the numbers

100 commits, every one made during the building period	**19,722** lines of Python in the backend package	**10,170** lines of TypeScript in the React app	**5,180** lines of tests
591 backend tests, all offline	**6** end-to-end browser tests of the built app	**74** HTTP endpoints, with streamed events	**85** design decisions, each recorded with its reason

1.4 What makes it different

Four things set Inkubit apart from a language model put in front of a simulator, and each is measured.

- **The model never has the last word.** Every model output passes a check that does not trust it: specs are validated, code is compared with an independent prediction, proposed circuits are proven equal, and numbers in reports are traced.
- **Proposals are checked, not trusted.** Discovery, learning, code design and the referee are loops in which a model proposes and an exact method decides. The model supplies the ideas; a proven verdict keeps them honest.
- **It reaches real systems.** An AI-discovered circuit beat the textbook circuit on a real IBM quantum computer. The satellite planner flies today's quantum-key satellites on their real orbits; the whole-paper lab reads a real 100-page paper.
- **Its own claims are held to the same rule.** A test rebuilds every number in the README and the submission from its data file, and this report is generated from the same data.

CHAPTER 2

The problem and the principle

Quantum programs fail silently, and language models are fluent but not exact. The answer is a division of labour: the models read, plan and propose, and exact physics decides what is true.

2.1 A wrong quantum program still runs

A reversed CNOT, or a qubit index off by one, does not crash a quantum program. It still runs and still prints a histogram; it is just the wrong histogram. On real hardware every histogram is noisy anyway, so a wrong answer can look like an ordinary bad day for the chip. Students and researchers sketch circuits and network layouts on paper long before they write code, and the step from a sketch to *correct* code is where the time goes. Errors also come from the notation itself: textbooks write states as kets, papers use their own diagrams, and frameworks such as Qiskit, Cirq, Quil and OpenQASM disagree on conventions such as bit order.

The obvious fix today is to ask an AI assistant. The assistant writes plausible code, explains it confidently and predicts the output. The prediction is the weak point. A model predicting a circuit's output is doing the calculation in its head, and that calculation grows exponentially with the number of qubits.

2.2 Language models are fluent, not exact

Inkubit measured this before it relied on it (Chapter 15). Muse Spark was asked to predict circuits' outcome odds on its own, the way a chat assistant answers, and was scored against exact simulation. It was exact on 13 of 13 textbook circuits and on 10 of 11 new small circuits; the easy version of the claim, that chatbots get quantum circuits wrong, did not hold. On bigger new circuits of 6 to 8 qubits it was exact on only 3 of 6. One answer described a near-uniform distribution with full confidence and put 43% of the probability in the wrong place. Nothing in the answer itself warned the reader.

> THE FINDING THAT SHAPED THE DESIGN
>
> A model's confidence says nothing about its accuracy on a problem it can't calculate. A checker that recomputes the answer exactly is not optional, and it must be independent: it must never read the model's code or reasoning, only the specification of what was meant.

2.3 The principle: the AI proposes, exact physics has the last word

Every feature of Inkubit follows one division of labour. The models do the parts that need perception, language and imagination. Exact methods do the parts that need correctness. Checks that trust neither side sit between them and the user.

INPUTS	pen sketch · camera on a whiteboard · typed problem · speech · paper (PDF or arXiv id) · figure from a paper · notation (Dirac, QASM, Quil, Cirq ...)
META MODELS	segment the drawing (SAM 3.1) · read it into a spec · plan a tool graph · propose layouts and codes · write and repair code · transcribe speech · write the report
EXACT ENGINES	density-matrix simulation · Qiskit Aer · L-BFGS optimiser · symplectic code checker · matching decoders · SGP4 orbits · STO-3G integrals · Lindblad pulses
CHECKS	spec validation · shot-noise bound · second simulator to 1e-6 · physics invariants · proofs of equality · number tracing · paired statistics
WHAT YOU SEE	verified result or "Not verified" · the model that said it · which numbers are traced · error bars

2.4 Rules the app keeps

A principle only matters if the software enforces it. These rules are enforced in code and covered by tests.

- **Every model answer names its model.** The badges say *Meta live*, *Mock* or *Mock fallback*, with the error. Offline mode never pretends to read a photo it doesn't know; it says that reading one needs the live models.
- **No faked success.** The repair loop stops when the repairer returns the same code twice. After four attempts the result is labelled **Not verified**, and its code and counts are marked as unverified.
- **Thresholds come from statistics, not from tests.** The verification bound is derived from the multinomial standard error, and it has never been loosened to make a test pass.
- **Every number is traced.** In the assistant and in mission reports, each number the model writes is matched, at the precision written, against the tool results. Untraced numbers are shown as untraced.
- **Speed-ups and replays are labelled.** The demo video tags any sped-up wait on screen. Micius, which re-entered the atmosphere in January 2026, flies only as a labelled replay of its 2017 experiments.
- **Our own documents are checked.** `tests/test_claims.py` rebuilds each quoted number from its data and fails if a document says anything else.

2.5 Who it is for

The start page offers nine use cases, each with a pen sketch on paper in the hand of the sample sketches. Each opens the part of the app that does that job: sketch to code, debugging, chemistry, quantum security, optimisation, satellite networks, cryptography, real hardware and trust. Students can learn from it by drawing, educators can turn a whiteboard into a live simulation, and researchers can check a paper's numbers or a chip's behaviour. The checks are the same for everyone.

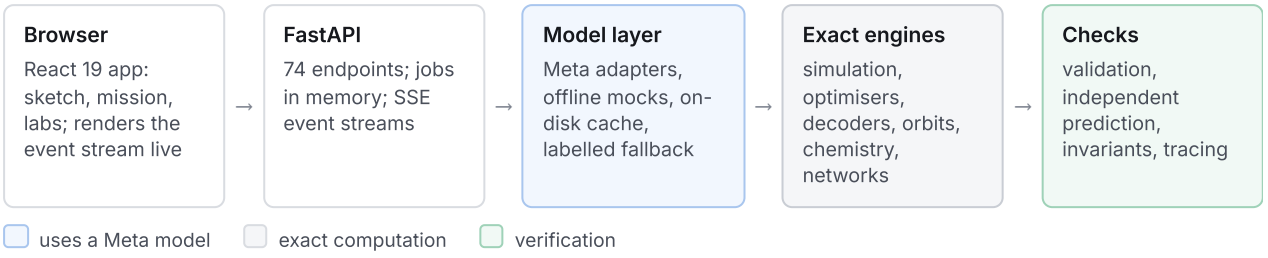
CHAPTER 3

Architecture

A Python backend of exact engines behind a FastAPI server, a React app that streams every step live, and a model layer through which every Meta model call passes. That layer has an offline twin, a cache and an honest fallback.

3.1 Overview

Inkubit is a single web application. The browser sends a request: a photo, a spec, a mission, a question or a paper. The server starts a job and returns at once. The job's events (plan, step started, attempt failed, repaired, verified) stream back over server-sent events, so the user watches the work happen instead of waiting for a result. There is no database: jobs live in memory, share links carry the whole spec in the URL, and per-viewer progress such as challenge stars is kept in the browser.



The model layer and the exact engines never call each other directly. Orchestration code such as the agent loop, mission control or a lab's driver asks the model for a proposal, hands it to an engine and passes the result through the checks. That separation is why the model can be swapped for an offline mock without changing anything downstream.

3.2 The backend

The backend is one Python package, `inkubit`, of about 19,722 lines. It runs on Python 3.13 with Qiskit 2.6 and Qiskit Aer, NumPy and SciPy, pydantic v2 for every schema, PyMatching for the decoders and the `sgp4` package for orbits. The main parts:

PART	WHAT IT DOES
`spec/`	The versioned circuit and network specification (pydantic, `extra="forbid"`), readable validation that reports every problem with its path, the JSON schema, and the flat draft schema the models fill
`adapters/`	One interface per model job, with Meta and mock implementations, the response cache and the fallback policy
`codegen/`, `sim/`	Deterministic template code, the static safety check, noise presets and the sandboxed subprocess runner
`verify/`, `agent/`	The independent expected distribution, metrics and verdicts; the plan → run → verify → diagnose → repair loop, fault injection and the event bus
`insights/`	Gate-by-gate states, entanglement, error budgets, Bell tests, tomography, zero-noise extrapolation, chip routing
`network/`, `orbits.py`, `satellites.py`	The physical link model, bottleneck search and purification; SGP4 propagation, ground stations on WGS84, pass finding and cloud weighting

PART	WHAT IT DOES
`mission.py`, `copilot.py`	Mission control (planning, parallel execution, repair, verification, traced reports) and the tool-calling assistant
`discovery.py`, `learning.py`, `codes.py`	The discovery lab, the learning loop and the stabilizer-code checker
`paper.py`, `reproduce.py`, `referee.py`, `arena.py`	Whole-paper and figure reproduction, the AI referee, the AI-alone benchmark
`decoder.py`, `pulse.py`, `estimator.py`, `distributed.py`	Surface-code decoders, the transmon pulse simulator, resource estimates, the distributed computer
`chem.py`, `qkd.py`, `qaoa.py`, `shor.py`, `qec.py`	The labs: H_2 from first principles, BB84, QAOA Max-Cut, Shor's algorithm, the error-correction lab
`rosetta.py`, `notation.py`	The Quantum Rosetta: exact parsers and writers for Dirac notation and quantum languages
`mlverify.py`, `observatory.py`, `precomputed.py`	The ML verifier, the simulation observatory, and the stored input-free results

3.3 The frontend

The app is React 19 with TypeScript, built by Vite, about 10,170 lines in 48 components. The start page leads with mission control, and a switch puts the quick assistant in the same place. Everything else sits in five sections that mount only when opened, so heavy requests do not start on page load. The open section is kept in the URL.

SECTION	WHAT IS IN IT
Top of the page	Mission control, or a quick question to the same exact tools
Sketch to code	Photo or camera, words or voice, the live board, the Quantum Rosetta, the use-case cards
Discover	Discovery lab, an AI scientist that learns, when does quantum win, reproduce a paper (a figure or a whole paper)
Hardware and networks	Pulse lab, ML decoder, code lab, error-correction lab, distributed computer, satellite planner
Labs and games	Challenges, bug hunt, Shor, QAOA, H_2, BB84 with a spy
Evidence	Simulation observatory with the ML verifier, AI alone vs Inkubit, the AI referee, the real IBM run

Visual rules are simple and kept everywhere. The colours are a charcoal ground, one blue for measured data, one orange for predictions, green and red only for pass and fail, and gold reserved for entanglement. The Bloch spheres are drawn in 3D as SVG and share one camera. Every chart has a crosshair readout, a legend and a "Show the numbers" table, and motion stops under `prefers-reduced-motion`. A guided tour (`/#tour`) walks through the highlights in under two minutes. It starts a real mission and a real discovery run early and comes back to their results.

3.4 The model layer

Every model job (read a sketch, write code, repair it, explain, plan a mission, propose layouts) is an interface with two implementations. The **Meta** implementation calls `https://api.meta.ai/v1` using only the endpoints and fields in Meta's documentation; the pages used are listed in the repository. The **mock** implementation is deterministic and offline. It recognises the bundled sample sketches by file hash and returns their recorded readings. It writes code from the template, repairs with a rule-based repairer, explains from templates filled with the real numbers, and refuses anything it can't do honestly.

- **Cache.** Responses are cached on disk by request hash, so repeated demos and runs cost nothing and are reproducible.

- **Fallback.** With `INKUBIT_ALLOW_FALLBACK=1`, a failed live call is answered by the mock, and the badge says *Mock fallback* together with the error.
- **Public mode.** With `INKUBIT_PUBLIC=1` on a shared host, code written by a model is never executed (the agent runs Inkubit's own template code). Billed calls stop at 300 per day, counted only on cache misses, after which the labelled offline models answer.

3.5 Jobs, events and streaming

Long work runs as a job with an event bus. Each step (a plan, an attempt, a verdict, a diagnosis, a repair, a tool result, a check) is a plain-language event that streams to the browser as a server-sent event. Mission control runs independent steps on a thread pool; each change is an event, and the browser draws it on a live graph. Slow results that need no input, such as the observatory's simulations and the pulse lab's sweep and DRAG calibration, are computed once and stored with the app. A test checks that they equal a fresh computation, so a cold start on a small container does not spend minutes recomputing them.

3.6 Technology

LAYER	CHOICE	WHY
Language	Python 3.13	Qiskit 2.6 and Aer ship wheels for it; the system Python was too old
Quantum	Qiskit 2.6, Qiskit Aer 0.18	Transpiler, Aer density-matrix simulation, OpenQASM 2/3, the independent second simulator
Numerics	NumPy, SciPy	Closed-form channels, optimisers, the neural networks (no ML framework is needed)
Decoding	PyMatching	Minimum-weight matching at distance 5, checked against an exact matcher at distance 3
Orbits	sgp4 (Vallado)	The standard propagator for CelesTrak elements; checked against Skyfield in a test
Server	FastAPI, Uvicorn, sse-starlette	Typed endpoints and event streams
Schemas	pydantic v2	One source for validation, the JSON schemas the models fill, and API types
Documents	pypdf	Cutting a paper's pages for the second transcription pass
Frontend	React 19, TypeScript, Vite	A typed single-page app; the build is served by the backend
Testing	pytest, Playwright	Offline backend tests; end-to-end tests that drive the built app in Chromium
Hosting	Docker, Koyeb free tier	512 MB and a tenth of a CPU are enough in public mode

CHAPTER 4

Meta's models in Inkubit

Five Meta models, each used where it is strongest, each behind a check. The models perceive, plan, write and propose; they never produce a physical result themselves.

4.1 Five models, each where it is strongest

MODEL	ENDPOINT	JOBS IN INKUBIT	CHECKED BY
Muse Spark 1.3	`/v1/chat/completions`, JSON-schema output, tools	Chooses what SAM looks for; reads photo and boxes into a spec; turns words into specs; writes and repairs code; explains in 12 languages; answers questions from a run's facts; gives challenge hints; proposes shorter circuits, layouts and codes; plans, repairs and reports missions; drives the assistant's tool calls; transcribes figures and paper pages; plays the bug hunt; predicts alone for the AI-alone benchmark	Spec validation, independent verification, exact proofs, the code checker, number tracing
Muse Wonder 1.0	`/v1/chat/completions`, tools, `reasoning_effort: medium`	Deep mode of the assistant (up to six rounds of tool calls); deep mode of mission control; lists the experiments in a whole paper; challenges Muse Spark in the referee	Number tracing; exact simulation of every experiment and claim
SAM 3.1	`/v1/responses`, streamed	Finds every drawn part of a sketch or a whiteboard frame, one concept prompt per noun phrase, with confidences	The overlay shows which model found which box; the user can correct the spec
Muse Image 1.0	`/v1/images/generations`	Illustrates the final circuit or network	Always shown next to the exact Qiskit drawing
Muse Voice Transcribe 1.0	`/v1/asr/transcribe`	Speech for the assistant, mission control and "describe it"; biased toward quantum words	The transcript goes through the same validated path as typed text

Muse Spark is the default for nearly everything: it is fast, reads images, follows schemas and calls tools. Muse Wonder is Meta's most capable model, built for multi-step agentic work. It costs four to five times as much per token, so it is used only where depth pays: long tool chains, the whole-paper pass and the referee. Standard-mode model IDs are used, so judges' and users' content is not used for training.

4.2 Structured output as the contract

Every model step that feeds an exact engine uses structured output: a JSON schema in `response_format` that the answer must satisfy. The schemas are kept flat on purpose, with every field required and no tuples or optional objects, because that is the most reliable shape for constrained decoding. A draft is converted into a real specification and validated. If validation fails, the model is re-prompted once with the readable errors, for example `operations[2] (RX): target qubit 3 is out of range; this circuit has 2 qubits`.

The first live run, with fallback switched off so that nothing could hide behind the mock, found two problems that offline tests could not. The circuit draft schema allowed `MEASURE` as a gate but had no field for its classical bit. Muse Spark put the GHZ sketch's meters there, so the spec could

never validate; the schema and prompt were fixed. Hindi explanations came back in Latin letters; the prompt now names each language's script. The whole live run cost about $0.40.

4.3 Tool calling and number tracing

The assistant, *Ask Inkubit*, is Muse Spark with tool calling. It chooses among nine function tools, each a thin wrapper over an exact engine: circuit simulation, the optimiser, the H_2 molecule, BB84, QAOA, Shor's algorithm, network links, error correction and the Rosetta. It calls several in parallel when it needs several. The system prompt forbids working out a quantum result without a tool. Tool errors go back to the model as data, so it can fix its own arguments.

After the answer, every number in it is matched against the numbers in the tool results and in the question, at the precision written or as a percentage. Small counts and bitstrings are skipped. The page shows "n of n traced", or lists the numbers that were not traced. The first live answers traced completely: 3 of 3, 5 of 5 and 14 of 14. Muse Wonder's deep mode answered "plan a 1,200 km satellite key link, then check whether a spy on 20% of the photons would be caught" by calling the link planner and BB84 in parallel, in 8.9 s, with 13 of 13 numbers traced. Mission control applies the same tracer to its reports (Chapter 6).

4.4 Seeing: SAM 3.1 and set-of-marks prompting

Reading a sketch pairs the two models the way Meta's documentation recommends: Muse Spark interprets the scene and decides what to segment, then hands the concepts to SAM. Muse Spark looks at the photo and names one to four noun phrases ("box with letter", "filled dot", "circle with plus"). SAM 3.1 runs once per phrase with confidences switched on, and its token output is parsed into boxes scaled to the uploaded image. Muse Spark then sees the photo, a copy with numbered boxes drawn on it and the box list, and fills the draft schema, linking every operation to the box it came from.

SAM is trained on photographs, and pen strokes on paper sometimes give few or no masks. The draft schema therefore also asks Muse Spark for each element's own box on its 0–1000 grid. If an element has no SAM segment, Muse Spark's box is used at confidence 0.6 and labelled *seen by muse-spark-1.3* in the overlay. The overlay is never empty, and it never claims SAM found something it did not.

4.5 Voice and images

"Say it" records the microphone with Web Audio, downsamples to 16 kHz and encodes a mono 16-bit WAV in the browser, so the server needs no ffmpeg. The recording goes to Muse Voice Transcribe in push-to-talk mode with a keyword list (qubit, Hadamard, CNOT, GHZ, Shor, …) that biases recognition toward quantum words. In a live check, three synthesised clips played through a fake microphone were transcribed in 3 to 4 s and became the right GHZ, Shor and satellite specifications. Offline, the button uses the browser's own recogniser, and the server never invents a transcript.

Muse Image draws a clean illustration of the final design. It is always shown next to the exact Qiskit drawing, so an imperfect picture can never mislead. The nine use-case images were generated in one restrained style (dark charcoal, off-white lines, one gold accent, no text) by a cached script, at about one cent per image.

4.6 Reliability and cost

- **Truncated answers.** A cut-off JSON answer is a successful HTTP response, and the cache stores it. The retry therefore doubles the token budget and marks itself, which changes the request, while the first attempt stays identical and keeps its cache entry.
- **Documents.** PDFs go to the model as a file block with a base64 data URL. For a long paper, pypdf cuts out only the pages each experiment needs (Chapter 9).
- **Cost control.** Everything is cached by request hash. Public mode caps billed calls per day. Muse Wonder is opt-in except where its depth is the point.

4.7 An open-weight model on a laptop

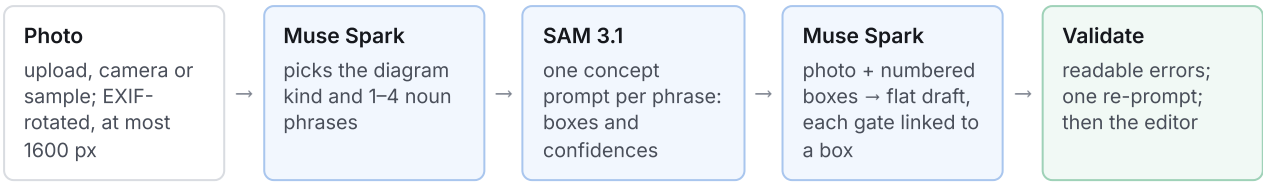
Meta's open-weight models were tried too. Muse Spark Open is an 816-billion-parameter mixture of experts, out of reach of an 8 GB laptop GPU. Muse Glimmer (27.9B, 4-bit, 18 GB with its vision projector) runs through Ollama's OpenAI-compatible endpoint with the same Meta client; only the base URL, the model name and a timeout change. It ran split 71% on the CPU and 29% on the GPU at about 1.6 tokens a second, so a reasoned answer took about five minutes. It answered the Bell pair exactly, under the JSON schema. That is too slow to drive the app, so the hosted models stay. A full AI-alone benchmark on Glimmer was stopped after one circuit when the machine ran low on memory, so no Glimmer results are claimed.

CHAPTER 5

Sketch to verified code

The original pipeline: draw a circuit on paper, and get running code whose result is proven right by an independent simulation of what you meant. When the code is wrong, the agent explains why, fixes it and runs it again, live on screen.

5.1 Reading a sketch



The reading path. The user sees the photo and the specification side by side and can correct either.

A sketch can be a circuit or a quantum network; Muse Spark decides which from the photo. The result is not trusted blindly. It is a draft that the user sees overlaid on the photo: hovering over a box highlights its line in the editor, and the editor validates as the user types. Words and speech take a parallel path. Muse Spark fills the same draft schema from a description such as "Bernstein-Vazirani with secret 1011" or "300 km of fiber with 2 repeaters". Offline, a deterministic parser handles named algorithms, gate lists and simple networks, and anything else gets a list of what it understands, never a guess.

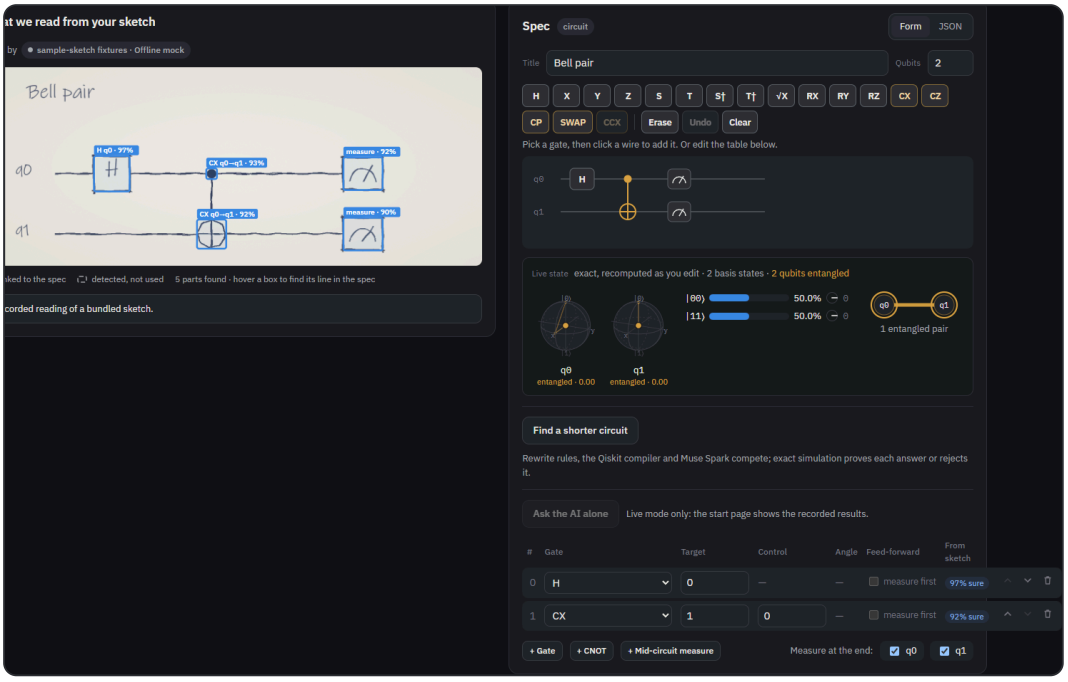


Figure 1 The photo with every part SAM 3.1 boxed, next to the specification Muse Spark read from it. Each operation is linked to its box, and validation runs as you type.

5.2 The specification

Everything downstream works from one JSON document, discriminated by `kind` and versioned. It is deliberately small: it is what the models fill, what the user edits, what the code is generated from, and what the verifier rebuilds the circuit from, independently.

```
{ "kind": "circuit", "version": "1.0", "num_qubits": 2, "num_clbits": 2,
  "operations": [
    {"gate": "H", "targets": [0], "source": {"bbox": [310, 250, 410, 350], "confidence": 0.97,
"segment_id": "s0"}},
    {"gate": "CX", "controls": [0], "targets": [1]} ],
  "measurements": [{"qubit": 0, "clbit": 0}, {"qubit": 1, "clbit": 1}],
  "metadata": {"title": "Bell pair", "origin": "image"} }
```

The gate set (H, X, Y, Z, S, T and their inverses, √X, rotations, CX, CZ, controlled phase, SWAP and Toffoli) is enough for Grover search, Bernstein-Vazirani, Deutsch-Jozsa, the QFT, phase estimation, teleportation and the W state. Measurements are terminal, which keeps the independent prediction exact. A circuit without a measurement is invalid, because there would be nothing to verify. Unknown fields are errors, not silently ignored typos. Common aliases such as `CNOT` are normalised, because hand labels and model output use them. Networks list ground stations, satellites and repeaters with optional memory coherence times and apertures, and fiber or free-space links with lengths and losses.

5.3 Writing and running code safely

Muse Spark writes a standalone Qiskit script for the specification; offline, a deterministic template does. In **demo mode**, the first attempt is broken on purpose with one of six realistic bugs, so that the repair is always visible: a missing CNOT, a reversed CNOT, a wrong qubit index, a missing measurement, an old import path, or a faked result that prints plausible counts without running anything.

Model-written code is treated as untrusted. Before it runs, the abstract syntax tree is checked: only `qiskit`, `qiskit_aer`, `json`, `math` and `numpy` may be imported, and calls such as `open`, `eval` and `exec` are rejected. The script then runs in a fresh, isolated Python process in a scratch folder, with a 60-second timeout and an environment from which every variable named like a key, token or secret has been removed, so generated code can never read the Meta key. This is a guard rail, not an operating-system sandbox, and the README says so. On the public host, model-written code is never executed at all.

5.4 Verification

The verifier never reads the generated code. It rebuilds the circuit from the specification with its own builder and computes the **exact, noise-aware** distribution the run should produce: a density-matrix simulation with the preset's gate noise, then the readout error applied exactly to each bit. Then it compares that prediction with the counts the code actually produced. A run passes only if both of these hold:

$$TVD(\text{measured}, \text{expected}) \le \max\left(\text{floor}, \tfrac{3}{2} \sum_i \sqrt{p_i(1-p_i)/N}\right) \text{ and } \text{Hellinger fidelity} \ge \text{minimum}$$

The second term is three standard deviations of sampling error on every outcome at once. For a Bell pair at 4,096 shots it is about 0.023, so honest runs essentially never fail. The floors (0.02, 0.03 and 0.04 for the ideal, low-noise and realistic presets) and the minimum fidelities (0.99, 0.985,

0.98) absorb the small differences between two independent constructions. Real faults are far outside these limits: a missing or reversed CNOT on a Bell pair gives a distance of about 0.46 to 0.5. In the submission's example, a correct Bell pair run was 0.004 from its prediction, against a limit of 0.040.

PRESET	1-QUBIT DEPOLARIZING	2-QUBIT DEPOLARIZING	READOUT 0→1 / 1→0	T1 / T2
ideal	0	0	0 / 0	none
low_noise	0.05%	0.5%	0.5% / 1%	none
realistic	0.1%	1.5%	2% / 4%	100 µs / 80 µs (50 ns and 400 ns gates)

KNOWN BLIND SPOTS, STATED IN THE APP

Verification compares measurement statistics. It cannot see a pure phase error that leaves the outcome distribution unchanged, and it does not catch changes smaller than the threshold, such as an angle off by a few degrees. The circuit tools in Chapter 13 (state-by-state inspection, process-fidelity proofs, Bell tests and tomography) look further than the verifier does.

5.5 Diagnosis and repair

When a run fails, the agent explains why in plain words, from the statistics alone. Correlations that should exist but don't suggest a missing or reversed entangling gate. Wrong marginals suggest a gate on the wrong qubit. A single outcome where noise should spread the counts suggests faked results. For example:

> *"Attempt 1 failed: expected ~50% |00⟩ and 50% |11⟩, got 49% |00⟩ and 48% |01⟩. Bits c0 and c1 should agree every time but come out independent; the entangling gate looks missing or points the wrong way. Repairing..."*

The repairer (Muse Spark live, a deliberately limited rule-based repairer offline) gets the full evidence: the specification, the code, the error, the verifier's verdict, the diagnosis and the history of earlier attempts. Every attempt keeps its code and a unified diff, so the interface shows exactly what each repair changed. There are at most four attempts. If the repairer returns identical code, the loop stops and reports failure instead of burning attempts. A run that cannot be fixed is reported as **Not verified**, and its code and counts are labelled as unverified.

5.6 The planted-fault benchmark

`scripts/benchmark.py` runs every gallery circuit under realistic noise, once clean and once with each fault that can be planted in it. It runs offline (template code and the rule-based repairer), so it measures the agent loop and the independent verifier, not a language model. **55 of 55 runs ended correctly**: every clean run verified on the first attempt, all 37 repairable faults were verified after one repair, and all 9 faked results were caught. The rule-based repairer deliberately has no rule for faked results, which is how the honest-failure path is demonstrated and tested. Muse Spark gets the same evidence and can fix that fault too.

CIRCUIT	CLEAN	IMPORT	NO MEASURE	WRONG QUBIT	REVERSED CX	NO CX	FAKE RESULT
Bell pair	✓ 1	✓ 2	✓ 2	✓ 2	✓ 2	✓ 2	caught
Bernstein-Vazirani (101)	✓ 1	✓ 2	✓ 2	✓ 2	✓ 2	✓ 2	caught

CIRCUIT	CLEAN	IMPORT	NO MEASURE	WRONG QUBIT	REVERSED CX	NO CX	FAKE RESULT
Deutsch-Jozsa (balanced)	✓ 1	✓ 2	✓ 2	✓ 2	✓ 2	✓ 2	caught
GHZ-3	✓ 1	✓ 2	✓ 2	✓ 2	✓ 2	✓ 2	caught
Grover search (2 qubits)	✓ 1	✓ 2	✓ 2	✓ 2	–	–	caught
Phase estimation of T	✓ 1	✓ 2	✓ 2	✓ 2	–	–	caught
Superdense coding (11)	✓ 1	✓ 2	✓ 2	✓ 2	✓ 2	✓ 2	caught
Superposition with a tilt	✓ 1	✓ 2	✓ 2	✓ 2	–	–	caught
Quantum AND (Toffoli)	✓ 1	✓ 2	✓ 2	✓ 2	–	–	caught

✓ n: verified after n attempts. Caught: the verifier refused a faked result. –: the fault cannot be planted in this circuit. Wall time 92 s.

5.7 Understanding the result

A verified result is only the start. Muse Spark explains it to a student using the real numbers, in any of twelve languages, each in its own script; offline, an English template is used and the page says so. Muse Image draws a clean diagram beside the exact one. The user can then ask questions about the run ("why did attempt 1 fail?"). Muse Spark answers only from that run's facts and says when the facts don't contain the answer. Any design can be shared as a link that carries the whole specification in the URL, so nothing is stored on a server, or downloaded as a one-file HTML report.

CHAPTER 6

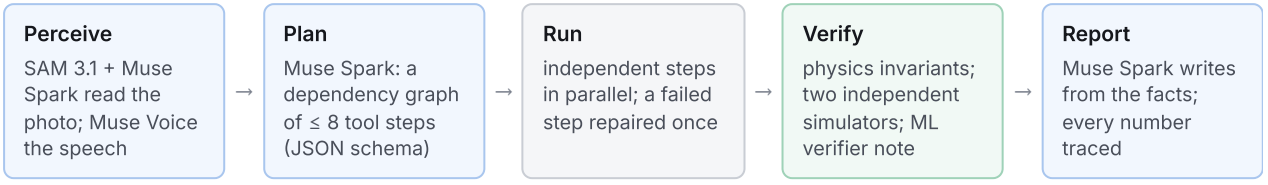
Mission control

One request, typed, spoken or photographed, becomes a plan of exact tools that runs live in parallel, repairs itself, verifies every step and writes a report in which every number is traced.

6.1 From one request to a plan

A mission can combine words, a photo and speech: "connect two cities 1,200 km apart with quantum-secure keys, compare satellite and fiber, and check whether a spy on 25% of the photons is caught". A photo goes through the sketch pipeline (SAM 3.1 and Muse Spark into a validated specification). If it is not a drawing, it goes to the Rosetta's notation reader instead. Muse Spark, or Muse Wonder in deep mode, then plans up to eight steps under a JSON schema. Each step has a tool, arguments (checked against that tool's own schema), dependencies and a stated purpose.

Circuits pass between steps by reference: an argument `circuit_from` names an earlier step, or the photo. A proven shorter circuit from the optimiser can therefore feed the ML verifier without the model copying gates, which is where models make transcription mistakes.



The mission loop. The live graph in the app follows the same five columns.

6.2 Running in parallel

Before anything runs, the plan is checked: unknown tools are dropped, and every dependency must point to an earlier step, which rules out cycles. A thread pool then runs every step whose dependencies are done. Each change is an event that streams to the browser and lights up the graph. A step that fails gets one repair: Muse Spark sees the error and the tool's schema and returns new arguments. Live runs in the interface took 12 to 23 seconds for missions such as two cities, photo to hardware, a molecule at three bond lengths, RSA, and exam scheduling.

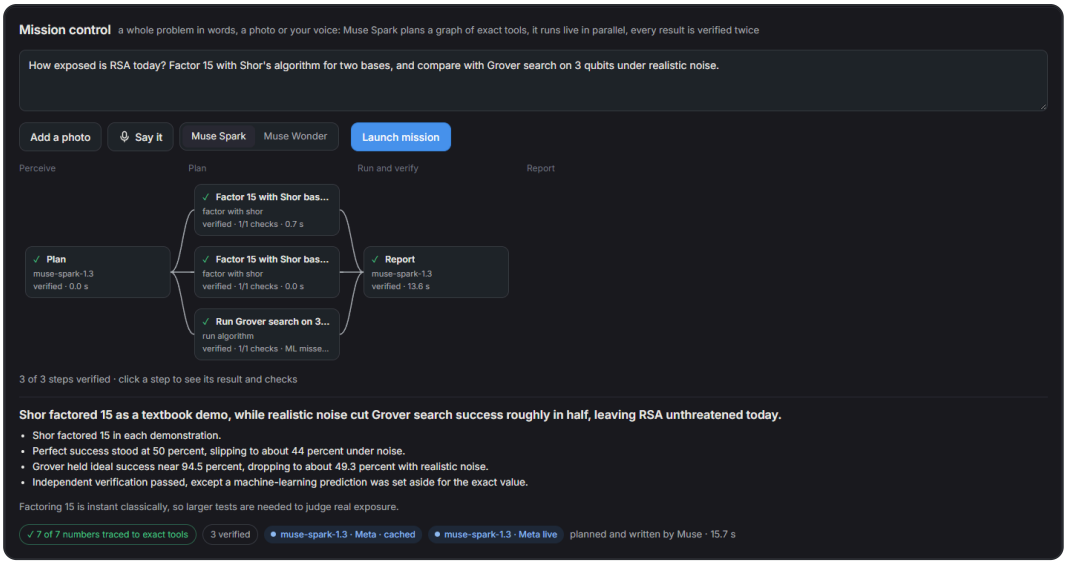


Figure 2 Mission control, live. Muse Spark planned three steps (two Shor runs and a Grover search) that ran in parallel and were each verified. The report's seven numbers were all traced to exact tools.

6.3 Verification that does not trust the tools

The tools are exact, but the checks assume they could still be wrong, through a bug or arguments a model chose badly. Every tool has physics invariants:

- the variational principle for H_2: no computed energy below the exact ground state;
- QAOA never above the brute-force best cut;
- Shor's factors must multiply back to the number;
- link fidelity in [¼, 1], and no key above the 11% error rate;
- for circuits, two independent noisy simulators that must agree to 1e-6: Inkubit's closed-form channels and Qiskit Aer's density matrix.

The ML verifier (Chapter 15) also runs on every circuit. A miss there grades the model, not the result, so it appears as a note and never flags the step.

6.4 Reports with every number traced

Muse Spark writes the report from the step facts and the text of their checks. The tracer then matches every number in the report against the tool results at the precision written. A number counts as traced only if a tool computed it; rounding 2,089 to 2,090 is not traced, by design. A finished mission downloads as one self-contained page: the plan, every step with its results, checks and arguments, the report with each number marked green (traced) or amber (not traced), the model calls, and standalone Qiskit code for every circuit.

6.5 The 30-mission benchmark

`scripts/mission_bench.py` ran 30 varied missions live, with Muse Spark planning and reporting. Three are hard on purpose: a number the Shor tool cannot factor, a problem with no quantum angle, and an ambiguous request.

30 / 30	68 / 68	204 / 209	15 s
missions finished with a report; 30 of 30 plans written by Muse Spark	steps verified (1 repaired by the model first)	report numbers traced to a tool result	median per mission, live; slowest 26 s

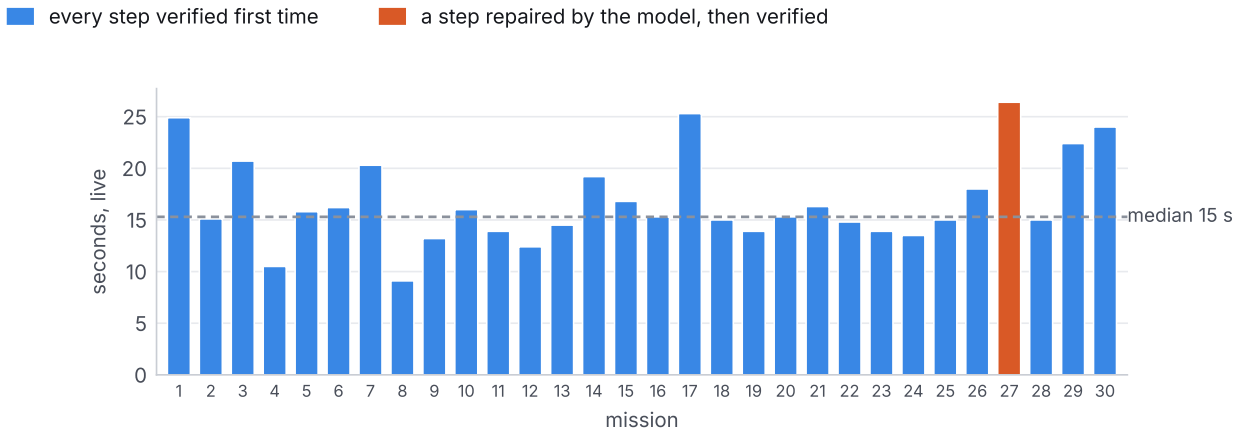


Figure 3 Time per mission in the live benchmark. 0 steps were flagged by a physics check and 0 failed or were skipped.

The five untraced numbers are the model rounding a tool's value (2,089 written as 2,090, twice, in two missions) or writing 6.4e-18 as "6.4 × 10^-18". The tracer flags a number unless it matches a tool result at the precision written, rather than guess. The hard cases were answered honestly. "Factor 21" ran the Shor tool and reported that the demonstration factored 15, not 21. "Sort a million numbers" reported that quantum search does not sort a list. "Do something interesting with entanglement" became four verified steps.

#	MISSION (AS TYPED)	STEPS	TRACED	S
1	Compare the hydrogen molecule at 0.6, 1.0 and 2.0 angstrom and say where the classical shortcut fails.	3	9/9	25
2	Is a spy listening to 40% of the photons caught? And one listening to 10%?	2	8/10	15
3	Factor 15 with Shor's algorithm using the bases 2, 7 and 13 and compare how often each run succeeds on noisy hardware.	3	6/6	21
4	Plan a 300 km fiber link and a 1,500 km satellite link; which gives more secret key per second?	2	7/7	10
5	Make a 5-qubit GHZ state, find a shorter equivalent if there is one, and check how well it survives realistic noise.	4	8/8	16
6	Run 3-qubit Grover search and the swap test; which one is more fragile under realistic noise?	4	6/6	16
7	Seven friends with feuds Ann-Ben, Ben-Cat, Cat-Dan, Dan-Eve, Eve-Fay, Fay-Gus, Gus-Ann: split them over two tables and say how well QAOA does.	1	7/7	20
8	When does error correction start to help, for today's gates and for gates ten times better?	2	4/4	9
9	Turn the state (|00⟩ + |11⟩)/√2 into a circuit and check how it survives noise with the ML verifier.	2	5/5	13
10	Run Simon's algorithm and Bernstein-Vazirani and tell me whether their answers survive realistic noise.	4	9/9	16
11	How does the energy of H2 at its equilibrium bond length compare between Hartree-Fock, the exact answer and noisy VQE with mitigation?	1	10/10	14
12	A bank wants to link two data centres 80 km apart with quantum keys. Plan the link and check an eavesdropper on 25% of photons.	2	10/10	12
13	Run the quantum Fourier transform period-finding example and the phase estimation example; do they still give the right answer with noise?	2	5/5	14
14	Build a W state on 3 qubits, then check whether a shorter circuit exists and how robust it is.	3	5/5	19
15	Teleport a qubit and swap entanglement between two pairs; which protocol loses more to noise?	2	6/6	17

#	MISSION (AS TYPED)	STEPS	TRACED	S
16	Schedule four exams, Maths, Art, Music and Biology, with clashes Maths-Art, Art-Music, Music-Biology, Biology-Maths, over two days.	1	5/5	15
17	What does the circuit h q[0]; cx q[0],q[1]; cx q[1],q[2]; measure all produce, and how entangled is it?	1	8/8	25
18	Plan a quantum network link of 2,000 km by satellite and of 2,000 km of fiber with 3 repeaters.	2	9/9	15
19	Compare a half adder circuit and a Toffoli AND circuit under realistic noise.	4	6/6	14
20	Simulate the 3-spin quantum magnet example and tell me how much the noise changes its outcome odds.	2	9/9	15
21	Is the Deutsch-Jozsa example still decisive on realistic hardware? Check it with the ML verifier too.	2	6/6	16
22	Which is worse for H2 at 2.5 angstrom: the Hartree-Fock error or the raw noisy VQE error?	1	11/11	15
23	Check BB84 with no eavesdropper and with one on every photon; what happens to the key?	2	6/8	14
24	For a 600 km fiber link, what is the weakest part and the best fix?	1	4/5	14
25	Run Grover on 2 qubits and on 3 qubits and compare their success under noise.	2	5/5	15
26	Explain how the bit-flip code example corrects an error and whether it survives noise.	3	8/8	18
27	Translate the Quil program H 0; CNOT 0 1 into a circuit and simulate it.	2	8/8	26
28	Factor 21 with Shor's algorithm.	2	6/6	15
29	Use a quantum computer to sort a list of a million numbers faster.	2	3/3	22
30	Do something interesting with entanglement.	4	5/5	24

CHAPTER 7

Discovery: AI proposes, physics proves

Models are good at guessing structure, and exact methods are good at judging it. The discovery lab combines the two to find shorter circuits than the textbook, and the learning loop shows the model getting better only when it sees exact physics' verdicts.

7.1 The discovery lab

The task is concrete: prepare a target state on a real chip's wiring with as few CNOTs as possible. Two-qubit gates carry about fifteen times the error of one-qubit gates on today's hardware, so every CNOT saved matters. The search has three stages:

1. **Propose.** Muse Spark proposes CNOT layouts under a JSON schema; pairs the chip cannot do are dropped, and a layout is credited to the model when it wins. Spanning trees of the coupling map and random layouts add more. Layouts are tried from the fewest CNOTs up.
2. **Fit.** For each layout, the single-qubit rotations between the CNOTs are fitted exactly by L-BFGS with restarts on a NumPy statevector (checked against Qiskit in a test). The fit is then pruned: rotations under 1e-3 are zeroed and the rest re-fitted.
3. **Prove.** A winner must reach an overlap of at least 1 − 1e-8 with the target in Qiskit's own `Statevector` of the rebuilt specification. The ML verifier screens the survivors, and exact noisy simulation decides between them.

Two baselines are compiled onto the same coupling map at Qiskit's highest optimisation level: the textbook circuit, and Qiskit's general `StatePreparation`. "Fewest found" is not a proof of minimality, and the page says so.

TARGET	CNOTS FOUND	BASELINES, COMPILED ONTO THE SAME WIRING
GHZ state on all 7 qubits of an IBM heavy-hex chip	**6**	textbook circuit 10; Qiskit StatePreparation 197
W state	**5**	Qiskit's compiler 11
Dicke state D(4,2)	**5**	Qiskit's compiler 11
A random 3-qubit state	**3**	Qiskit's compiler 4

Results quoted from the README's record of live runs; each winner proven by Qiskit to overlap 1 − 1e-8 with its target.

Six CNOTs is the least any circuit can use to entangle seven qubits into one connected group: a spanning tree of seven nodes has six edges. On heavy-hex wiring, not all of the textbook circuit's CNOTs fall on the chip's links, and routing adds four more. The lab found a layout that follows the chip's own tree. The comparison was then repeated on a real chip (Chapter 15): on seven qubits of IBM's ibm_fez wired the same way, Muse Spark's own layout reached a GHZ fidelity of 0.814 against 0.795 for the textbook circuit.

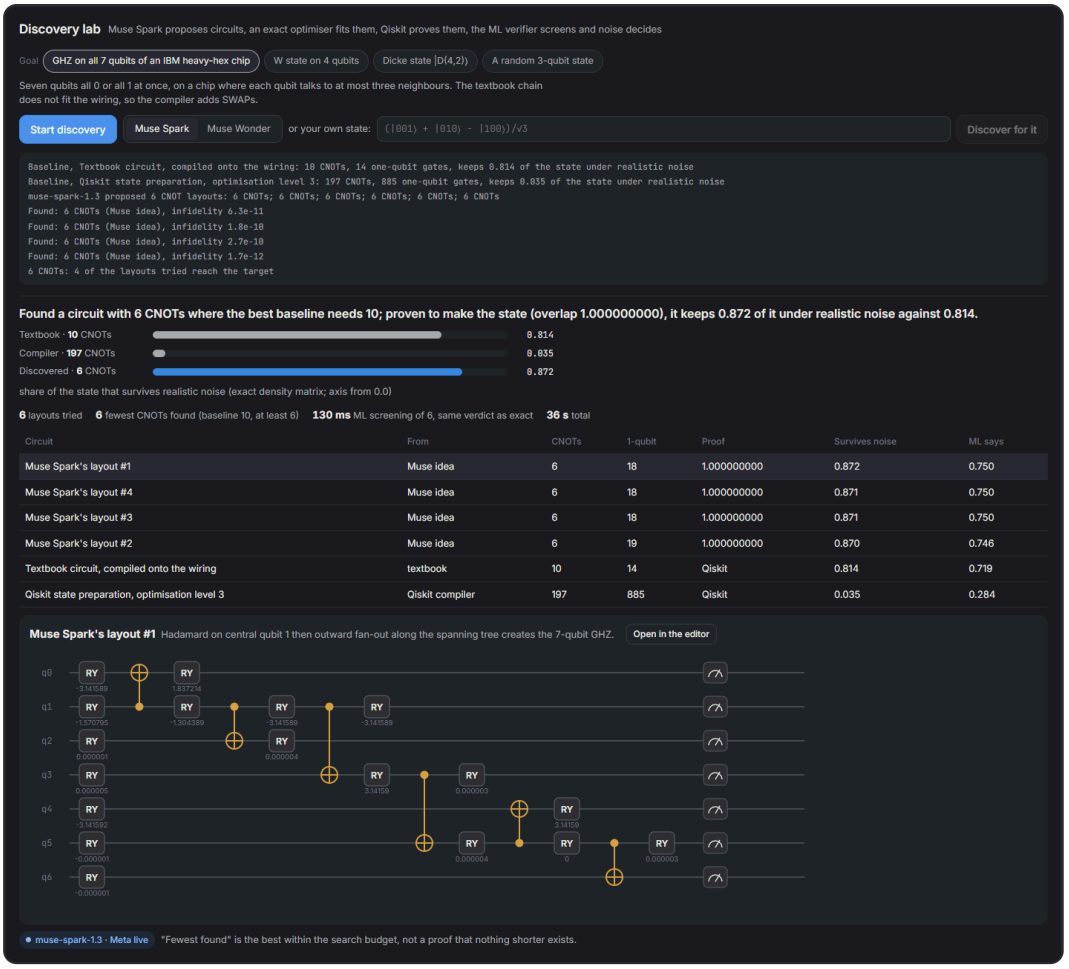


Figure 4 The discovery lab on a GHZ state across all 7 qubits of a heavy-hex chip: 6 CNOTs, proven by Qiskit, against 10 for the compiled textbook circuit.

7.2 An AI scientist that learns under proof

The discovery lab uses the model once. The learning loop asks a sharper question: *does the model get better when it sees what exact physics said about its earlier ideas?* Muse Spark proposes up to four CNOT layouts per round for a target state. The exact optimiser fits each one, and Qiskit proves every layout that reaches the target. Three conditions get the same number of proposals:

- **With feedback:** each round's prompt lists every earlier layout and its verdict;
- **Asked afresh:** the same model, asked again each round with no history (the prompts are numbered so no answer is reused from the cache);
- **Random wiring:** random allowed pairs with the same lengths as the feedback run.

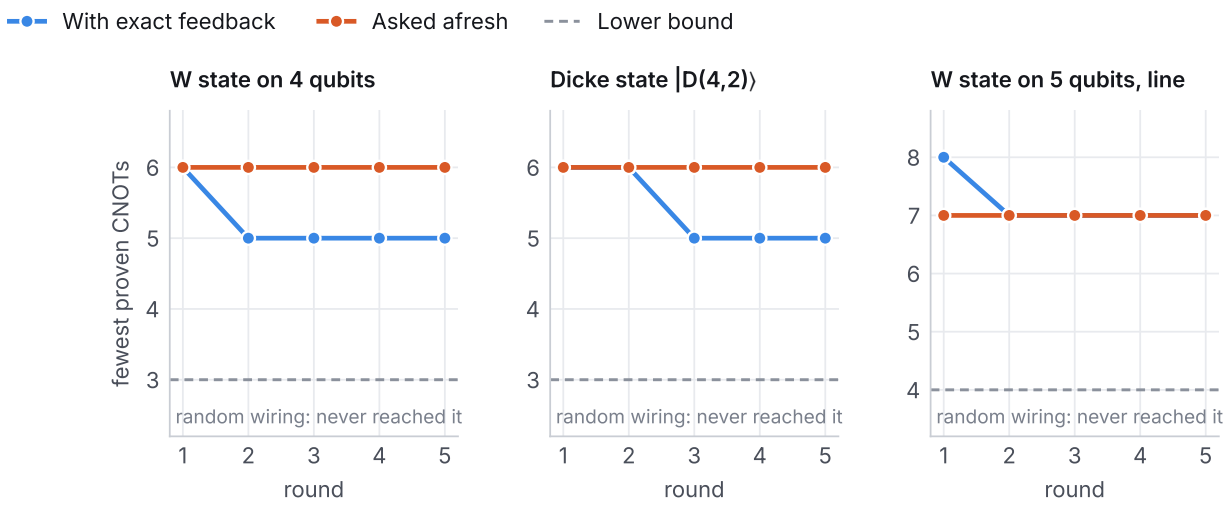


Figure 5 Fewest proven CNOTs after each of 5 rounds (muse-spark-1.3, live, about 10 minutes in all). With exact feedback the best proven circuit improved on every target; asked afresh it never improved on its first round. The dashed line is the lower bound from each state's structure.

TARGET	CONDITION	R1	R2	R3	R4	R5	REACHED
W state on 4 qubits	with feedback	6	5	5	5	5	10%
	asked afresh	6	6	6	6	6	10%
	random wiring	–	–	–	–	–	0%
Dicke state |D(4,2)⟩	with feedback	6	6	5	5	5	10%
	asked afresh	6	6	6	6	6	25%
	random wiring	–	–	–	–	–	0%
W state on 5 qubits in a line	with feedback	8	7	7	7	7	15%
	asked afresh	7	7	7	7	7	35%
	random wiring	–	–	–	–	–	0%

7.3 What the curves show

With feedback, the best proven layout improved on all 3 targets (6 to 5, 6 to 5 and 8 to 7 CNOTs). Without feedback the model never improved on its first round, and it repeated itself. On the five-qubit line its first answer was already 7, so asking again could not help. Random wiring of the same lengths never reached a target at all, which shows that the model's layouts carry real structure.

The reach rates tell the other half of the story. With feedback, fewer of the model's layouts worked than without it (10–15% against 10–35%). The model aimed below its own best and missed more often, and that is how it found the shorter ones. This is the behaviour of an experimenter, not of a lookup table: with the evidence in front of it, the model takes risks. Without it, it plays safe and stays where it was.

> ENGINEERING NOTE
>
> The first full run died on a truncated answer. A cut-off JSON answer is a successful HTTP response and is cached like any other, so a plain retry would have returned the same truncation. The retry now doubles the token budget and marks itself, which changes the request, while the first attempt stays identical and keeps its cache entry.

CHAPTER 8

AI-designed error-correcting codes

Muse Spark designs stabilizer codes, and an exact checker proves what each one can do. For hardware where phase flips dominate, it found a code that needs 9 phase-flip errors to fail, 3 times the surface code's 3.

8.1 Stabilizer codes in brief

A quantum error-correcting code spreads k logical qubits over n physical ones. A stabilizer code is described by $n - k$ commuting Pauli operators, its generators, written as strings such as `XZZXI`. Measuring them reveals errors without disturbing the stored information. The code's distance d is the smallest number of single-qubit errors that can change the logical information without being detected. A code with $d = 3$ corrects any single error, and its parameters are written $[[n, k, d]]$.

Real hardware is often **biased**: some qubits suffer phase flips (Z errors) far more often than bit flips (X errors). For such hardware, what matters is d_Z, the smallest number of Z errors alone that defeats the code, as long as the code still corrects any single error of any kind.

8.2 An exact checker

`inkubit/codes.py` checks any stabilizer code exactly, in the symplectic picture where each Pauli string is a binary vector:

- the generators must **commute** (symplectic inner product zero for every pair) and be **independent** (full rank over GF(2)); then $k = n -$ rank;
- the **distance** is found by brute force over every Pauli error from weight 1 up: the first error that commutes with every generator but is not itself in the stabilizer group is a logical error, and its weight is d;
- d_X and d_Z are the same search restricted to X-only and to Z-only errors.

The textbook codes are scored by the same checker, so the model's codes are judged on the same scale:

KNOWN CODE	[[N,K,D]]	DX	DZ	CSS
Five-qubit perfect code	[[5,1,3]]	5	5	no
Steane code	[[7,1,3]]	3	3	yes
Shor code	[[9,1,3]]	3	3	yes
Rotated surface code, d = 3	[[9,1,3]]	3	3	yes
XZZX surface code, d = 3	[[9,1,3]]	3	3	no

8.3 What Muse Spark designed

For each target, Muse Spark proposed up to three codes per round over 4 rounds, and saw every earlier verdict in its next prompt (`scripts/codes_bench.py`). The first two targets ask for the best possible distance in five and in seven qubits (the second only with CSS generators, all-X or all-Z).

Muse Spark found the five-qubit perfect code and a [[7,1,3]] CSS code in its first round. Both are optimal, so there is no curve to show. Some later proposals were invalid; the checker said why (wrong length, for instance), and they never counted.

TARGET	BEST PER ROUND	BEST CODE FOUND (GENERATORS)	VERDICT
One logical qubit in 5 physical qubits	3 → 3 → 3 → 3	`XZZXI IXZZX XIXZZ ZXIXZ`	valid [[5,1,3]], dX = 5, dZ = 5
A CSS code: 1 logical qubit in 7	3 → 3 → 3 → 3	`XXXXIII XXIIXXI XIXIXIX ZZZZIII ZZIIZZI ZIZIZIZ`	valid [[7,1,3]], dX = 3, dZ = 3, CSS
Biased noise: 1 logical qubit in 9	3 → 9 → 9 → 9	`YXIIIIIII IXYZZZZZZ ZZYXIIIII IIIXYZZZZ ZZZZYXIII IIIIIXYZZ ZZZZZZYXI IIIIIIIXY`	valid [[9,1,3]], dX = 7, dZ = 9

The third target is the interesting one. Its standard answers in round 1 had $d_Z = 3$, like the surface code. In round 2, Muse Spark proposed a [[9,1,3]] code with $d_Z = 9$ and $d_X = 7$. A logical error caused by phase flips alone needs all 9 qubits to flip, 3 times the surface code's 3, while any one-qubit error of any kind is still corrected. Later rounds found more codes with the same parameters, but none better.

8.4 Reading the result honestly

The checker proves the distances; they are facts about the generators, not estimates. Distances are not the whole story, though. How well a code performs under realistic biased noise also depends on the decoder, the circuits that measure the generators and the error rates. Inkubit does not yet simulate a decoder for the new code, so the claim is limited to what the checker proves. Muse Spark's own explanation of its design, "adjacent-pair X parts form a repetition code giving Z distance 9 while step-pattern Z parts keep X and Y checks distinct", is shown as the model's idea, next to the checker's verdict. The page lets anyone enter their own generators and get the same exact analysis.

CHAPTER 9

Reading the literature

Muse Spark and Muse Wonder read figures and whole papers; they transcribe and never compute. Exact simulation then judges every stated number against its error bar, and the lab says which experiments the paper does not specify completely instead of guessing them.

9.1 Reproduce a figure

Upload a figure from a paper. Muse Spark transcribes the circuit as OpenQASM, which the exact parser reads, with the sketch pipeline as a fallback. It also transcribes the claimed outcome probabilities, the shot count, the hardware and the bit order. The circuit is re-run exactly and every claim is judged against an error bar:

- with a known shot count, three standard deviations of shot noise (at least 0.01);
- without one, 0.03 for hardware results and 0.005 for ideal ones; a hardware claim may also sit between Inkubit's ideal and realistic predictions;
- if the bit order is not stated, both orders are tried, and the better fit is used and labelled as inferred.

In a live check, Muse Spark read a printed figure's circuit, all eight bar heights, the shot count and the bit order in 8 seconds, and every claim was judged within its error bar.

9.2 Reproduce a whole paper

A figure is one experiment; a paper is many. Give Inkubit an arXiv id or a PDF. One pass over a hundred-page paper, with Muse Wonder reading the whole PDF, proved fragile: it listed repeated runs of one Bell circuit, dropped numbers into its notes, and once returned truncated JSON. Two passes work:

1. **Find.** Muse Wonder reads the whole paper and lists up to eight experiments on *different* circuits, each with the pages it is on.
2. **Transcribe.** pypdf cuts those pages, plus the next one, into a small PDF. Muse Spark transcribes each experiment on its own, in parallel, under the figure checker's schema.
3. **Judge.** Circuits go through the exact parser, and every stated probability through the figure checker's judge.

On arXiv:1804.03719, *Quantum Algorithm Implementations for Beginners*, a 100-page paper, the lab listed 8 experiments in 107 s live. 5 had a complete circuit and stated odds; 3 of those were reproduced, and 11 of the 19 stated probabilities checked out. 3 experiments were reported as not completely specified on their pages, instead of being guessed.

EXPERIMENT	PAGE	HARDWARE	VERDICT
Bell state preparation	15	qasm_simulator	reproduced
Grover search for x*=(1,1)	19	ibmqx4	not completely specified

EXPERIMENT	PAGE	HARDWARE	VERDICT
Bernstein-Vazirani, secret 01	23	ibmqx4	not completely specified
HHL, |b>=|0>, Z measurement	29	IBM QX Simulator	not completely specified
Shor's algorithm for N=15 and x=11	33	ibmqx4	not reproduced
Hadamard test: regular representation of second element of S2/A2, state |0>	38	IBM's 5-qubit computer	reproduced
Hadamard test irreducible rep second element S2/A2 state |1>	38	IBM's 5-qubit computer	reproduced
Discrete-time quantum random walk, 4 steps on 4-vertex cycle	46	ibm_vigo	not reproduced

9.3 What "not reproduced" means

The two experiments not reproduced are Shor's algorithm for 15 and a quantum walk. Their stated numbers are measurements from 2018-era hardware, whose noise Inkubit's realistic preset does not reach. That is a statement about the preset and the old devices, not an accusation against the paper. The lab reports what it can check, with what error bars, and leaves the interpretation visible.

9.4 A silent error that a test caught

Building the whole-paper lab exposed a quiet bug elsewhere. The QASM extractor stopped at the first statement it did not recognise, so a circuit containing an opaque gate became an empty circuit. An empty circuit gives P(000) = 1, and a paper claiming a high P(000) was then "reproduced". The extractor now refuses any statement that acts on qubits and that it cannot read, and it strips comments first. A wrong reproduction is worse than an honest "cannot check", and the test suite now holds that line.

CHAPTER 10

Error correction and learned decoders

From the three-qubit bit-flip code to a distance-5 surface code. A neural decoder beats minimum-weight matching at distance 3; at distance 5 the bigger code suppresses errors, and a network on top of matching fixes what matching gets wrong. Every gain is tested on the same shots, with a paired test.

10.1 The error-correction lab

The simplest code spreads one logical qubit over three. Each qubit flips with probability p, then the decoder (two CNOTs and a Toffoli, syndrome measurement and correction in deferred form) restores the majority. With perfect gates the logical error is $3p^2 - 2p^3$; the lab simulates it and checks it against that formula. With realistic gates, the code's own gates put a floor under the logical error, so the code beats a bare qubit only above a **break-even** error rate: 10.8% with today's gates, falling to 0.87% with gates ten times better. Switch the noise to phase flips and the same code makes things worse than no code at all. That is the threshold idea in one picture, and the reason real codes protect against both kinds of error.

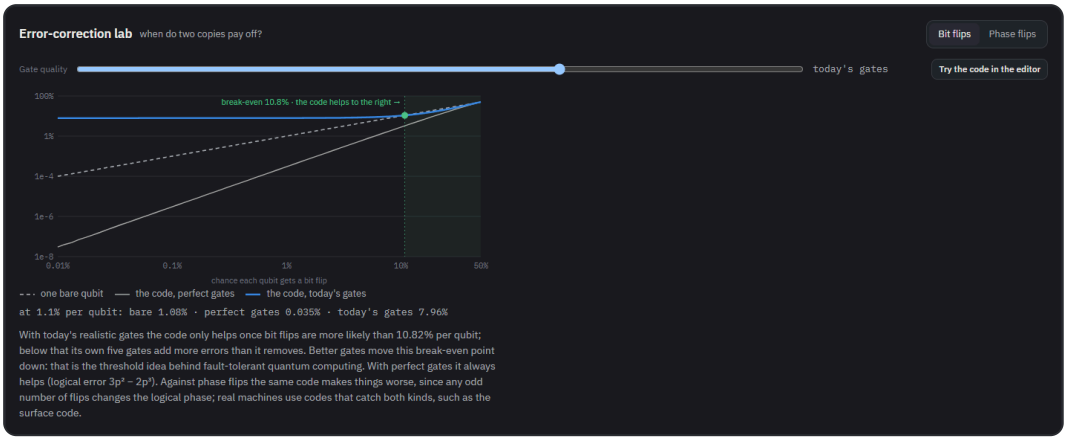


Figure 6 The error-correction lab: the bit-flip code against a bare qubit on log-log axes, with the break-even point moving as the gate quality slider changes.

10.2 The surface code and its decoders

The surface code is the leading candidate for large quantum computers. Data qubits sit on a grid, and X-type and Z-type checks measure their parities every round. A decoder reads the pattern of changed checks (the *detection events*) over several rounds and decides which correction to apply. Inkubit simulates a rotated surface code in a Z-basis memory experiment with phenomenological depolarizing noise. Sampling is exact in NumPy. The standard baseline is **minimum-weight perfect matching**: pair up detection events along the most likely error chains in the space-time graph, using log-likelihood weights. At distance 3 Inkubit uses its own exact matcher (dynamic programming over the events); at distance 5 it uses PyMatching on the same graph.

Matching has a known weakness. It decodes X and Z errors separately, but a Y error is both at once, and it shows up in both kinds of checks. Learned decoders, such as Google DeepMind's AlphaQubit, beat matching partly by seeing such correlations.

10.3 Distance 3: a network beats matching

The distance-3 network is small (25-128-128-1, ReLU). Its input is every detection event of both check types, plus the noise level. It was trained on 1.8 million shots at six noise levels and tested on 200,000 new shots per level. At 1% noise it makes **21% fewer logical errors** than matching (0.72% against 0.92% over three rounds). The gain holds at every one of the 8 tested noise levels, and it exceeds both 95% intervals.

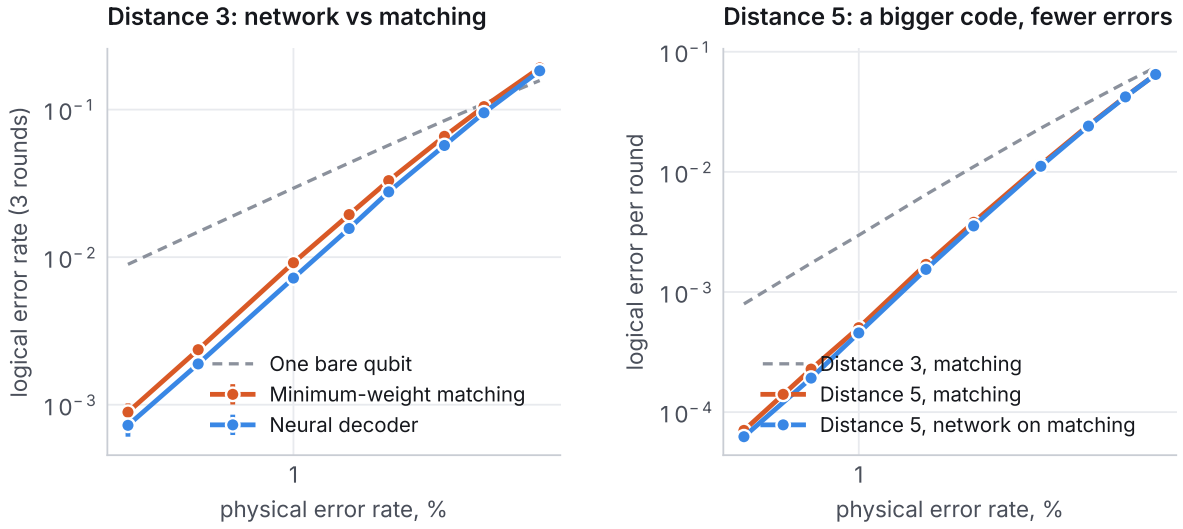


Figure 7 Left: logical error at distance 3 (3 rounds), with 95% intervals. Right: logical error per round at distances 3 and 5, and the distance-5 network built on matching.

10.4 Distance 5: a bigger code, fewer errors

`layout(d)` generates the rotated surface code for any odd distance. It reproduces the distance-3 lists exactly. At $d = 5$ it gives 25 data qubits with 12 X and 12 Z checks, all commuting, of rank 24, leaving one logical qubit. Before PyMatching was trusted, it was run against the exact matcher on 20,000 distance-3 shots and agreed on all of them. The detectors had to be ordered time-major: with check-major order the two agreed on only 80%, which is how the correct order was pinned down.

The per-round logical error ε comes from memory experiments of d rounds, through $1 - 2P = (1 - 2\varepsilon)^{\text{rounds}}$. At 1% noise the per-round error falls from 3.0×10^{-3} at distance 3 to 5.0×10^{-4} at distance 5. The suppression factor $\Lambda = \varepsilon_3/\varepsilon_5$ is **5.9**. It is 11.3 at 0.5% noise and shrinks to 1.15 at 6%, as the code approaches its threshold. For scale, Google's Willow chip measured $\Lambda = 2.14$ on real hardware in 2024. That comparison is for orientation only: Willow's number includes circuit-level noise from real gates and measurements, which the phenomenological model here leaves out.

10.5 A network on top of matching

The first distance-5 network saw the events of both kinds plus matching's verdict, and learned only to copy matching: +0% at every noise level. The second also sees *where* matching put its X corrections, and where a second matching, on the X checks, put Z corrections. A qubit corrected by both is a Y error, which neither matching can see on its own. This 172-256-256-1 network was trained on 12 million fresh shots, none seen twice. At 1% noise it makes **10% fewer logical errors** than matching.

Both decoders run on the same shots, so the honest test is paired. A McNemar test counts only the shots on which they disagree. At 1% the network fixes 123 shots that matching gets wrong and breaks 27 that matching gets right (z = 7.8). The gain is significant at 8 of 9 noise levels; the exception is 6%, near threshold.

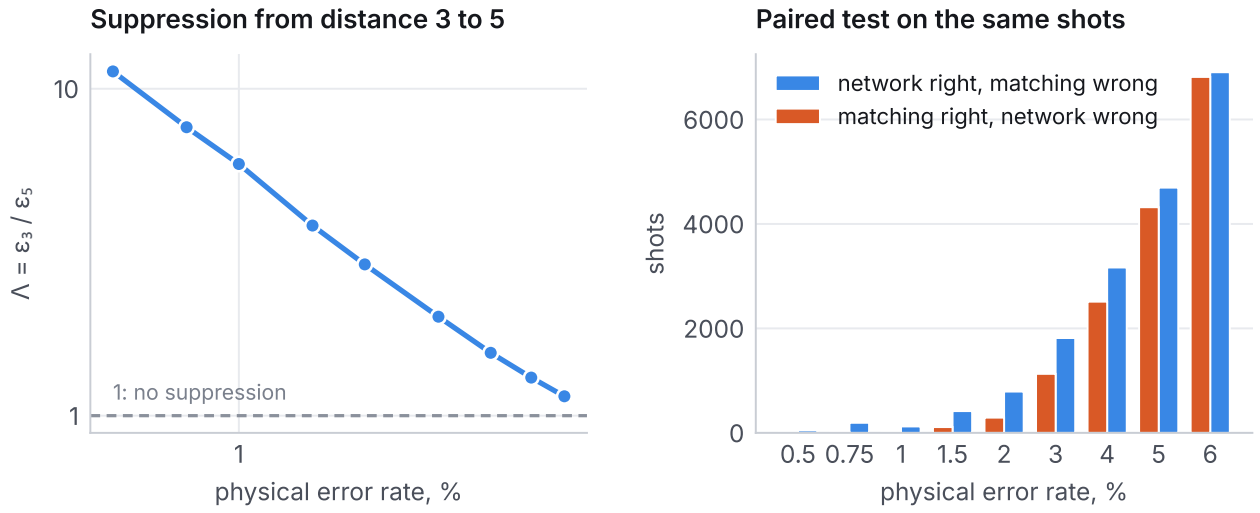


Figure 8 Left: the suppression factor Λ from distance 3 to 5 at each noise level. Right: on the same shots, how often the network was right where matching was wrong, and the reverse.

P	E PER ROUND, D = 3	E PER ROUND, D = 5	Λ	NETWORK	FIXES / BREAKS	PAIRED Z
0.5%	8.0×10^{-4}	7.0×10^{-5}	11.30	6.2×10^{-5}	48 / 8	5.3
0.75%	1.7×10^{-3}	2.3×10^{-4}	7.63	1.9×10^{-4}	193 / 13	12.5
1%	3.0×10^{-3}	5.0×10^{-4}	5.89	4.6×10^{-4}	123 / 27	7.8
1.5%	6.5×10^{-3}	1.7×10^{-3}	3.82	1.5×10^{-3}	417 / 111	13.3
2%	1.1×10^{-2}	3.8×10^{-3}	2.90	3.5×10^{-3}	791 / 292	15.2
3%	2.3×10^{-2}	1.2×10^{-2}	2.01	1.1×10^{-2}	1816 / 1130	12.6
4%	3.8×10^{-2}	2.4×10^{-2}	1.56	2.4×10^{-2}	3166 / 2512	8.7
5%	5.5×10^{-2}	4.2×10^{-2}	1.31	4.2×10^{-2}	4695 / 4320	3.9
6%	7.4×10^{-2}	6.5×10^{-2}	1.15	6.5×10^{-2}	6904 / 6814	0.8

CHAPTER 11

Down to the hardware

Below the circuit is the microwave pulse; beside one chip is a second one; ahead is the question of when any of this beats a classical computer. Inkubit simulates all three from first principles.

11.1 The pulse lab

An X gate on a superconducting qubit is a shaped microwave pulse. A transmon is not really a two-level system: it has a third level, and its second transition sits only 300 MHz below the qubit's own in this model. A fast pulse leaks population into that level and picks up a phase error from it. The pulse lab simulates a three-level transmon in the rotating frame, with T1 and T2 from the realistic preset as Lindblad terms. Each pulse is integrated exactly with a 9 × 9 superoperator, and the result is the average gate fidelity of the qubit-subspace channel, with leakage counted as error.

Two pulse shapes are compared. A truncated **Gaussian** π pulse has σ = T/4. **DRAG** adds a quadrature component −β·dΩ/dt / α that cancels the leakage and phase errors to first order. β is calibrated by a golden-section search and lands near 0.5, the first-order optimum for phase errors.

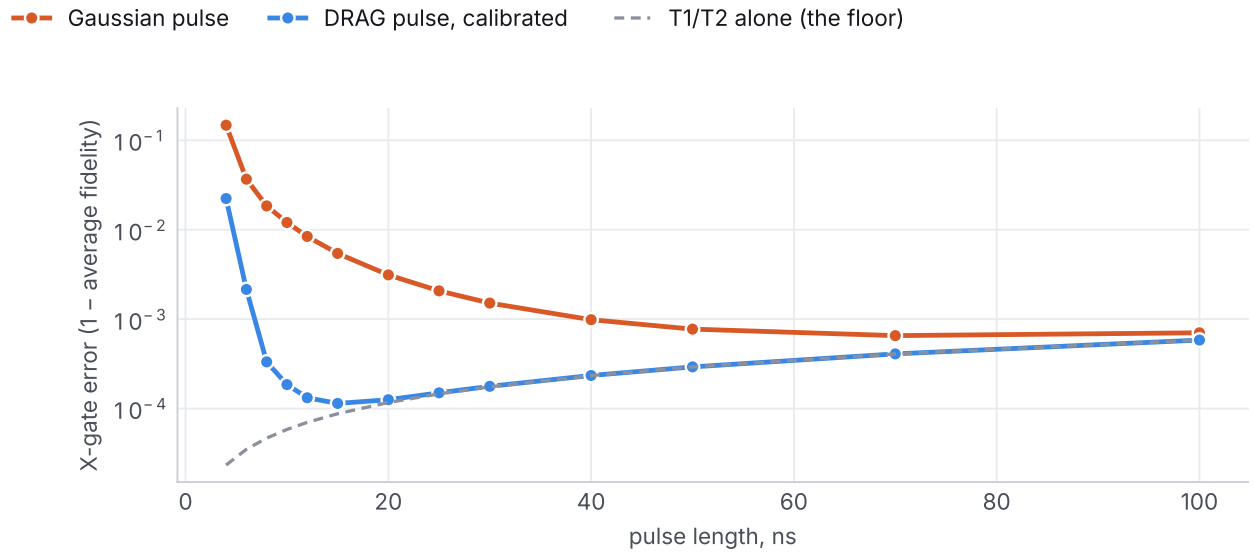


Figure 9 X-gate error against pulse length. Short Gaussian pulses leak into the third level; calibrated DRAG reaches the T1/T2 floor at a fraction of the length.

At the shortest lengths, leakage dominates. By 15 ns a Gaussian pulse's error of 5.4×10^{-3} is mostly the phase error the third level induces; leakage is only 4.4×10^{-5} of it. DRAG's quadrature cancels both. DRAG reaches 1.1×10^{-4} at 15 ns, where little but decoherence is left. The best Gaussian needs 70 ns to reach 6.5×10^{-4}, and it is limited there by the decoherence that its own length adds. This is why every real superconducting device uses DRAG or a descendant of it.

11.2 A distributed quantum computer

One way to build bigger machines is to join small chips with links. The distributed computer runs one circuit on two chips joined by a link from the network planner (Chapter 12). An exhaustive search finds the split with the fewest remote CNOTs. Each remote CNOT is done by gate teleportation (Eisert et al., 2000), which consumes one of the link's Bell pairs. With a Werner pair of fidelity F, the ideal CNOT is followed by an X on the target, a Z on the control, or both, each with probability $(1 - F)/3$. This follows from where Pauli errors on the pair propagate, and a test checks it. Readout flips on the ancillas add more X and Z errors, the two local CNOTs are charged their depolarizing error, and every data qubit idles under T1/T2 while the messages travel.

The score is process fidelity over all inputs, computed by running the circuit on halves of Bell pairs, which limits it to five qubits. State or outcome fidelity were rejected: a relaxed qubit can be steered back to the right answer by later gates, which would flatter the link. The page shows fidelity against link length from 10 m to 100 km, against one chip, and against long-lived memory qubits.

11.3 When does quantum win?

Describe a problem in words or by voice. Muse Spark maps it to factoring, unstructured search, physics simulation or "no known speedup". A surface-code resource estimate then meets the best classical route. The quantum model follows Beverland et al. (2022):

- logical error per patch per cycle $P_L = 0.03\,(p/0.01)^{(d+1)/2}$, with $2d^2$ physical qubits per patch;
- routing overhead $2Q + \sqrt{(8Q)} + 1$ patches for Q logical qubits; one logical step per d cycles; a 1% total failure budget;
- magic-state factories of 25 patches producing one T state every $10d$ cycles, in two 15-to-1 distillation rounds.

The algorithm costs come from the literature: Gidney and Ekerå (2021) for factoring; Grover with a stated oracle cost for search; and first-order Trotter for a 2D Ising magnet, with the rigorous $\|[A,B]\| \le 8n$ bound and Ross–Selinger rotation costs (a worst case, said on the page). The classical side uses the general number field sieve scaled from RSA-250 by its L-function, N/2 checks for search, and the exponential memory wall for simulation.

RSA-2048 ON SUPERCONDUCTING HARDWARE	INKUBIT MODEL	GIDNEY & EKERÅ (2021)
Code distance	31	–
Logical qubits	6,190	–
T gates	1.05×10^{10}	–
Magic-state factories	49 (9% of the qubits)	–
Physical qubits	26.6 million	20 million
Runtime	18.5 hours	8 hours

The model lands within 1.3× (qubits) and 2.3× (time) of the published figure, and every factoring result on the page shows this comparison.

The estimator says "classical wins" when classical wins. Brute-force search of a 64-bit key, for example, costs Grover so much error correction per oracle call that a classical cluster finishes first. Saying "not yet" is part of being trustworthy about quantum computing.

CHAPTER 12

Quantum networks and satellites

The same canvas plans entanglement-distribution networks from physics, finds their bottleneck and fixes it. The satellite planner flies the quantum-key satellites in orbit today on their real orbits, under the real cloud forecast.

12.1 The link model

The network model is analytic and pure Python, and each number in it is a stated assumption:

- **Fiber:** 0.2 dB/km at 1550 nm, light at 2×10^5 km/s, detectors 80% efficient, dark clicks 10^{-7} per gate.
- **Free space at 810 nm:** the diffraction spot of the satellite's transmit aperture ($1.22\,\lambda/D$), broadened by 8 μrad of turbulence and pointing jitter, against the ground telescope's aperture; 4 dB of atmosphere per pass and 8 dB of optics and coupling. For the 1,100 + 1,600 km dual downlink this gives about 66 dB and 0.8 pairs per second, in line with Micius's published 64–82 dB and about one pair per second over 1,200 km. It was calibrated to that public result, not to the tests.
- **Pairs:** 10^8 pulses per second, 10% pair probability per pulse, intrinsic Werner fidelity 0.98. Accidental coincidences from dark and background clicks lower the visibility, so long links lose fidelity as well as rate.
- **Repeaters** herald at L/c with 100 multiplexed modes and store pairs in memories whose Werner parameter decays as exp(−t/T2). Swapping multiplies Werner parameters, times 0.99 per swap. $F = (3w + 1)/4$, and the error rate is $(1 - w)/2$.
- **Key:** the BBM92 asymptotic rate, $R = ½\, R_{pairs}\, (1 - 1.16\, h(e) - h(e))$, clamped at zero.

12.2 Bottleneck search and purification

The network agent finds the bottleneck: memory decoherence if any stored pair waits longer than 0.3 T2, otherwise the lossiest link. It then tries every applicable fix as a new, validated specification and re-simulates each from scratch. The fixes are a midpoint relay, a rebalanced repeater spacing, a higher-elevation pass, a bigger telescope (up to 1.8 m, the Lijiang telescope used with Micius), better memories and entanglement purification. The best fix is kept if it improves the score by more than 1%, for up to three rounds. While there is no key at all, the score is pairs per second × $(F - ¼)$, so a fix that makes progress still counts.

Purification (BBPSSW) turns two noisy pairs into one better pair with some probability. It halves the pair rate, so pairs wait longer in memory. With poor memories it therefore makes things worse, and on a 600 km chain the agent learns to upgrade the memories first and only then purify, going from no key to 0.05 bit/s. The fiber chain preset goes from no key to 0.39 bit/s. The satellite preset, close to the 2017 Micius geometry, goes from 0.013 to 1.87 bit/s (pairs from 0.83 to 7.4 per second, fidelity from 0.86 to 0.94). After every search, sanity checks run: fidelity in [¼, 1], key no more than half the pair rate, no key above 11% error, and a longer weakest link must not raise the rate.

12.3 Satellites versus fiber

The simulation observatory uses the planner's own model to ask the basic question of quantum networking: when is a satellite worth it? The satellite sits 500 km above the midpoint between two cities. Slant range and elevation come from spherical geometry, and the curve stops where either station would see the satellite below 15°.

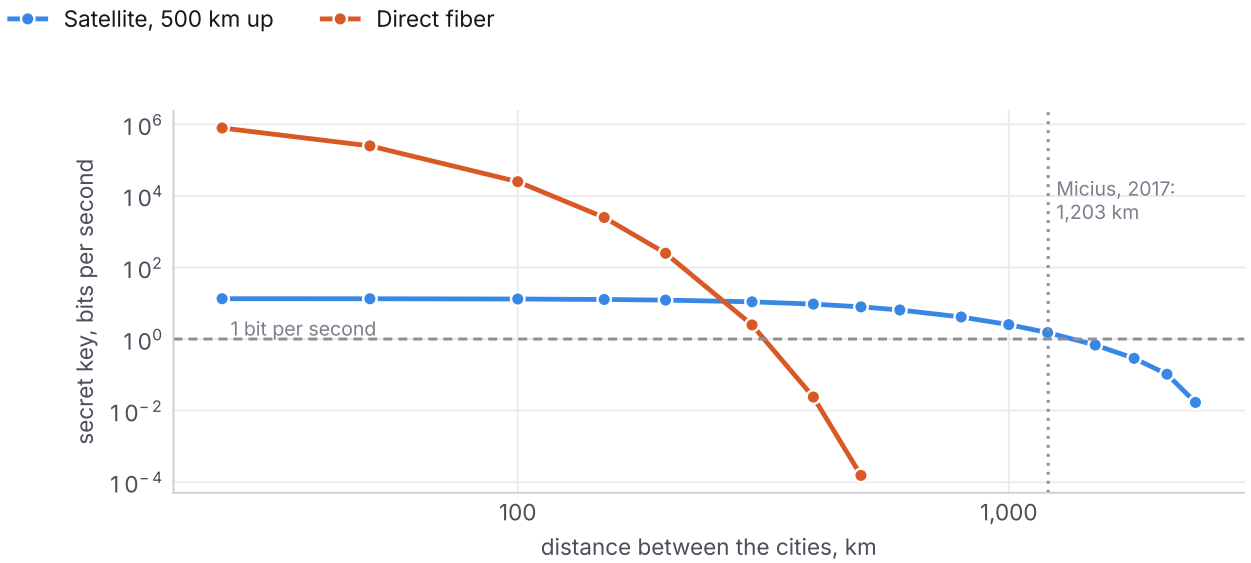


Figure 10 Past about 260 km the satellite beats fiber, and fiber gives no secret key at all beyond 500 km. At 1,203 km, the distance Micius bridged in 2017, the modelled satellite still gives 1.5 key bits per second, and 0.017 at 2,400 km.

12.4 Real satellites, live

The planner was first built around Micius. When it was time to swap the modelled orbit for real elements, CelesTrak's catalogue showed that Micius had re-entered on 25 January 2026, and Jinan-1 on 15 April 2025. The quantum-key satellites in orbit now are **SpeQtre** (NORAD 66769; RAL Space and SpeQtral; a 12U CubeSat with an entangled-photon source and an 80 mm telescope) and **QUBE-II** (NORAD 69023; a German 8U CubeSat, 85 mm, prepare-and-measure). Both now fly on their real orbits, and Micius stays only as a labelled replay of its 2017 experiments.

- **Elements:** CelesTrak TLEs, checksummed, cached for two hours, with a labelled fallback to the last good copy and then a shipped snapshot.
- **Propagation:** SGP4 (Vallado's code) gives TEME coordinates, and IAU-82 GMST rotates them into the Earth-fixed frame. Ground stations sit on the WGS84 ellipsoid with the geodetic vertical. Against Skyfield, an independent implementation, over a day of SpeQtre passes: elevation and azimuth agree within 0.0035°, range within 34 m. A test asserts 0.01° and 0.1 km.
- **Passes:** every night pass over two ground stations in the next 24 hours (above 15°, with the Sun below −12°), the key each gives through the satellite's own telescope (8 cm spreads the beam about 1.7 times wider than Micius's 30 cm), and a 7-day outlook.
- **Weather:** Open-Meteo's hourly cloud cover at each station scales each pass's key by the clear fraction (a first approximation: a low pass looks through more sky than the forecast's zenith).

Pass times, elevations and clouds are real data; key bits remain estimates, because the real payloads' source rates and protocols differ from the model. Tests never touch the network: CelesTrak and Open-Meteo are stubbed.

CHAPTER 13

Inside a circuit

Verification says whether a run is right. These tools show why: the state after every gate, who is entangled with whom, which noise source costs the most, whether a shorter circuit does the same job, and whether any classical explanation could fake the correlations.

13.1 Gate by gate

After every run, or before it with **Step through**, Inkubit computes from the specification alone the exact state after each gate. It shows the amplitudes, with phases relative to the largest amplitude (a global phase is unobservable), the state in Dirac notation, and a Bloch sphere per qubit. An arrow shorter than the sphere's radius means that qubit is entangled with others. The spheres are drawn in 3D and share one camera, so turning one turns all of them and they stay comparable.

Entanglement is drawn as a web. Each entangled pair gets a gold thread whose width is its concurrence (Wootters' formula, tested against Qiskit's), and each qubit's gold arc is its entanglement entropy. Group entanglement in which no pair is entangled on its own, as in a GHZ state, is drawn as a dashed hull and named, because concurrence alone would show nothing.

The same view carries the engineering numbers. The **error budget** gives the fidelity each noise source costs when it acts alone. A sweep finds how much better the hardware must be for a 99% match (a Bell pair needs about 7×). **Readout correction** inverts each bit's known confusion matrix. The **chip cost** compiles the circuit onto all-to-all, line and heavy-hex wirings and counts the extra SWAPs. These noisy analyses evolve the density matrix with the verifier's own error channels, built in closed form, and a test checks that they agree with Qiskit Aer to 1e-8. A **what-if** slider scales the noise from perfect to ten times worse and shows the exact distribution at each position, not an interpolation.

13.2 Shorter circuits, proven equal

Find a shorter circuit runs up to four optimisers: rewrite rules, the Qiskit transpiler at level 3, the two together, and (live) Muse Spark. Each rewrite rule comes with a reason a student can check:

- H·H = 1; S then S is Z; H on both sides turns CX into CZ;
- a CNOT whose control is still |0⟩ never fires; a CZ whose partner is |1⟩ is a Z on the other qubit (phase kickback);
- a diagonal gate right before measurement changes nothing a measurement can see; a SWAP before measurement is a relabelling of classical bits; gates that never reach a measured qubit can go.

No proposal is trusted. `prove()` compares each one with the original exactly and labels it with the strongest claim that holds: the *same gate* for every input (process fidelity, up to 8 qubits), the *same state* from |0…0⟩, or the *same measured odds*. Anything else is rejected, together with the outcome

where it differs. The thresholds are floating-point tolerances, not physics: $1 - F < 10^{-12}$, and a distance below 10^{-9} for the odds. A fuzz test applies the rules to 300 random circuits and requires every rewrite to keep the odds; 4,000 more passed locally.

Muse Spark sees the circuit, the measured qubits, its cost and the best cost so far, but never another optimiser's circuit. If its proposal is rejected, it gets one retry with the exact reason. On the two-qubit Grover example, whose answer is fixed in advance, Muse Spark noticed that the circuit only ever produces $|11\rangle$ and proposed two X gates. Exact simulation proved it, where rewrite rules plus Qiskit's level-3 transpiler had stopped at seven gates. When a result only flips bits, the panel says so: the circuit's answer was fixed in advance. Phase estimation of a known phase, for example, shrinks from 15 two-qubit gates to one X gate. That is true and worth knowing, but it is not a trick that works for unknown inputs.

13.3 Zero-noise extrapolation

Error mitigation lets today's noisy machines estimate what a perfect one would give. Inkubit first inverts readout error, then runs the circuit at 1×, 3× and 5× its gate noise by *folding*: U, then $U\,U^{-1}\,U$, then $U\,U^{-1}\,U\,U^{-1}\,U$. These have the same ideal result with three and five times the gates, and folding is what hardware can actually do. Every outcome's probability is then extrapolated back to zero noise with Richardson weights (15/8, −5/4, 3/8). With exact probabilities, a Grover search goes from a 0.882 match as measured to 0.999. The panel repeats the method with 4,000 sampled shots per noise level, because extrapolation amplifies shot noise: it needs about 5.2 times the shots for the same error bars. For some circuits that cost eats the gain, and the panel says so.

13.4 Bell tests and tomography

Is it really quantum? For every entangled pair, Inkubit computes the CHSH value under the best measurement settings, from the Horodecki closed form $S = 2\sqrt{(m_1 + m_2)}$ of the pair's correlation matrix. Classical theories allow at most 2, and quantum mechanics reaches 2.83. When exactly three qubits are entangled, it also computes the Mermin value: classical theories allow at most 2, quantum mechanics 4. That value has no closed form, so it is optimised from 16 fixed starts, and the panel says the settings were found, not proven optimal. Each value is computed three ways: ideal; on the noisy state with the settings planned for the ideal one, as an experiment would do; and from 4,000 sampled shots per setting, with readout error and a standard error.

- A Bell pair gives 2.828 ideal and 2.395 ± 0.025 on realistic hardware, 16 standard errors above the classical limit.
- A GHZ state has no entangled pair at all, yet reaches the Mermin maximum of 4.
- A W state's pairs are entangled but stay within the classical limit (1.886), while the trio reaches 3.046, the known value.

State tomography rebuilds the state the way a laboratory must, for up to three qubits. It measures along X, Y or Z in all 27 combinations (1,000 shots each, readout error included and then inverted), estimates every Pauli expectation, inverts to a density matrix and projects it onto the nearest physical state (Smolin, Gambetta and Smith, 2012). The result is drawn as two 3D cityscapes, the intended state against the rebuilt one, with the fidelity, the purity, and how much of the gap is hardware noise and how much is finite shots.

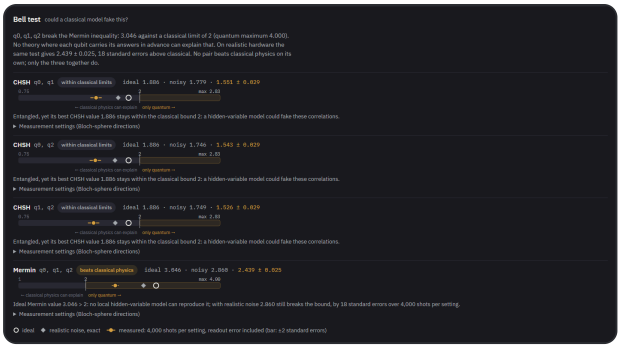


Figure 11 Bell tests: the W state's pairs stay classical; the trio does not.



Figure 12 Tomography of a GHZ state from 27 settings, drawn as a density-matrix cityscape.

13.5 The Quantum Rosetta

Quantum computing is written in several languages at once: Dirac notation, circuit diagrams, and five or more programming languages. Newcomers often stall on the notation before they reach the physics. The Rosetta translates between all of them, both ways, and checks every translation instead of trusting it.

- **Reading is exact** wherever a grammar exists. A recursive-descent parser reads Dirac notation (implicit products, $\otimes$, $\sqrt{}$, π, $e^{i\Phi}$, named states and simple LaTeX). Qiskit's importers read OpenQASM 2 and 3, and small line parsers read Quil and Qiskit gate calls. Muse Spark is used only for code without a parser here (Cirq, PennyLane, Q#, Braket) and for photos, which it transcribes under a schema with an instruction not to solve. The transcription then goes through the exact parser, and model readings are labelled.
- **A state gets a circuit** from the cheapest of several methods: one rotation per independent qubit, a CNOT fan-out for GHZ-like groups, a breadth-first search over textbook gates for groups of up to three qubits, Qiskit's state preparation, or a gallery circuit that makes the same state. The result must reach fidelity $1 - 10^{-8}$ by exact simulation, or the request fails.
- **Eight output languages** are written by fixed rules. The Qiskit, OpenQASM 2 and 3 and Cirq versions are executed and must give the same state, and the page says which were run.

Every step's sentence ("entangles qubits 0 and 1", "changes only a phase") is derived from the exact states before and after it, not generated by a model.

CHAPTER 14

Labs and games

Chemistry, cryptography and optimisation from first principles, and games that teach by being scored exactly. In each one the model helps, and exact physics keeps score.

14.1 Challenges

Eleven puzzles in three levels (warm-up, entanglement, gate golf) ask the player to build a target state or gate: a fair coin, a minus sign, a quarter turn, NOT without X, a Bell pair, a singlet, a hidden phase, GHZ, SWAP from CNOTs, CZ from one CNOT, and the W state. Each puzzle is defined by a reference circuit that never leaves the server. State puzzles compare $|\langle \text{target}|\psi\rangle|^2$ and gate puzzles compare the process fidelity $|\mathrm{Tr}(U_t^\dagger U)|^2/d^2$, so a global phase never counts and any correct construction is accepted. Solved means a fidelity of at least 0.999. Stars work like golf (solved, within par + 2, at or under par), and each reference solution is tested to reach exactly par.

Near misses get specific feedback, such as "the probabilities are already right; a relative phase is off" or "2 of 4 basis inputs already map correctly". Muse Spark gives hints: it sees the goal, the player's circuit, the exact score and the author's hint for that stage, never the solution, and it names at most one gate. A puzzle can be solved by clicking gates onto wires, by describing the answer in words, or by drawing it on paper and taking a photo.

14.2 Bug hunt

One gate in a known circuit is wrong: a wrong wire, control and target swapped, a look-alike gate, an angle slip or a stray gate. The player compares what the circuit should give with what it gave, and clicks the culprit against the clock. Rounds are stateless: (level, seed) regenerates the same round, so the page never receives the answer before the player guesses. Each planted bug must move at least 20% of the probability, so it can be seen in 1,000 shots.

Grading is by exact search, not by matching the planted index. Every one-gate edit (delete it, swap the gate, move it, flip control and target, change the angle) is simulated, and any gate whose edit restores the intended odds exactly counts. Some circuits have many such gates: in Shor's circuit, 22 of 28 gates counted. A round is therefore re-planted when too many gates would count. Live, Muse Spark plays the same round in parallel, from exactly what the player sees. In checks it found the bug in 4 of 4 rounds, in 9 to 24 seconds.

14.3 Split the party (QAOA)

Guests and feuds form a graph the player edits by clicking. QAOA seats the guests at two tables, which is the Max-Cut problem, the shape of many scheduling and routing problems. Brute force gives the true best seating; the depth-1 landscape shows every angle pair with the optimiser's path; depths 1 to 3 are optimised exactly. Checked: the 6-ring at depth 1 gives exactly ¾ of the best cut, the known value, and depth 3 solves it. Three layers find a best seating 95% of the time with perfect qubits and about half the time with realistic noise. The summary says plainly that at this size QAOA shows the mechanism, not a speedup.

14.4 H_2 from first principles

The molecule lab computes the hydrogen molecule without a chemistry package. The STO-3G integrals for two s functions have closed forms (Szabo and Ostlund, chapter 3), and they are tested against the book's values. By symmetry the problem reduces to one qubit, H = aI + bZ + cX, and the page says so. VQE measures Z and X under the realistic preset's noise; since that noise maps the Bloch vector affinely, three simulations fix every angle's expectation (checked to 10^{-5}), taking the lab from 110 s to under 4 s.

At 2.5 Å, where the electrons become correlated, Hartree-Fock is off by 233 mHa. Raw VQE on realistic noisy hardware is off by 21 mHa. Undoing the calibrated noise map, which is what calibration circuits measure on hardware, brings it to 0.8 mHa from the exact energy, against a shot-noise error bar of 2.1 mHa at 4,000 shots. The mitigated value lands within shot noise of exact at all 28 bond lengths.

14.5 Catch the spy (BB84)

Quantum key distribution with an eavesdropper. Random bits and bases, an intercept-resend eavesdropper on a share f of the photons, sifting, the error-rate check and privacy amplification run photon by photon, following the measurement rule (checked against Qiskit). Eve on a share f adds $f/4$ to the error rate, estimated from half the sifted bits. Above 11% the key is thrown away; below, it shrinks to $n(1 - 2h(\text{QBER}))$, so what remains is what Eve cannot know. At $f = 1$ she guesses 75% of the bits, and both facts are checked statistically.

14.6 The algorithm library

The gallery holds 31 circuits: three basics and 27 algorithms in 8 categories, each with its exact answer pinned in a test. The newest were written by a script that refuses to save a circuit that misses its answer derived on paper; on the way it caught two traps in Qiskit's library (a bit order and a hidden qubit permutation, D78). They include quantum counting, HHL ($A^{-1}b$ in the ratio 9 : 1), a Fourier-space adder, Shor's nine-qubit code and two nonlocal games: CHSH, won $\cos^2(\pi/8)$ of the time against a classical 3/4, and the GHZ game, won every time.

CATEGORY	CIRCUITS, EACH CHECKED AGAINST ITS KNOWN ANSWER (QUBITS)
Basics	Superposition with a tilt (2q), Bell pair (2q), GHZ-3 (3q), Active reset (measure, then fix) (1q)
Entanglement & communication	W state (one of three) (3q), Superdense coding (sends 11) (2q), Teleportation (deferred measurement) (3q), Teleportation (feed-forward) (3q), Entanglement swapping (4q), Swap test: how alike are two states? (3q)
Nonlocal games	The CHSH game (4q), The GHZ game (Mermin) (6q)
Oracles & hidden structure	Deutsch-Jozsa (balanced) (3q), Bernstein-Vazirani (secret 101) (4q), Simon's algorithm (secret 11) (3q), Hidden shift (bent function, s = 1011) (4q)
Search & counting	Grover search (2 qubits) (2q), Grover search (3 qubits, 2 rounds) (3q), Grover search with two marked items (3q), Quantum counting (2 marked of 4) (5q)
Fourier & phase	Phase estimation of T (4q), Quantum Fourier transform finds a period (3q), Shor: factor 15 with a = 7 (7q)
Arithmetic & logic	Quantum AND (Toffoli) (3q), Quantum half adder (4q), Draper adder: 5 + 6 in Fourier space (7q)
Linear algebra & simulation	HHL: solve a 2×2 linear system (4q), Simulating a quantum magnet (Trotter) (3q)
Error correction	Bit-flip code (an error on q1) (3q), Bit-flip code with feed-forward correction (4q), Shor's 9-qubit code corrects a Y error (9q)

CHAPTER 15

Evidence

Claims about AI and quantum hardware are easy to make and hard to check. This chapter collects the measurements behind Inkubit's design: how far a model alone can be trusted, what happens when two models debate, how good a learned shortcut is, and how the predictions held up on a real quantum computer.

15.1 AI alone vs Inkubit

The pitch that "a chatbot hallucinates and Inkubit does not" was tested, not asserted. The same model (muse-spark-1.3, medium reasoning effort) was given each circuit as data, with no title, no notes and no simulator. It predicted every outcome's probability under a JSON schema, the way a chat assistant would answer. Exact simulation scored each answer by total variation distance, the share of probability in the wrong place: *exact* below 1%, *close* below 5%, *wrong* from 5%. The sets were the textbook examples, which a model has seen many times; new seeded circuits of 3 to 5 qubits plus a six-guest QAOA circuit, which is what a student's own sketch looks like to a model; and bigger new circuits of 6 to 8 qubits, all put in superposition first, with 4 measured. The recorded run is dated 2026-10-31.

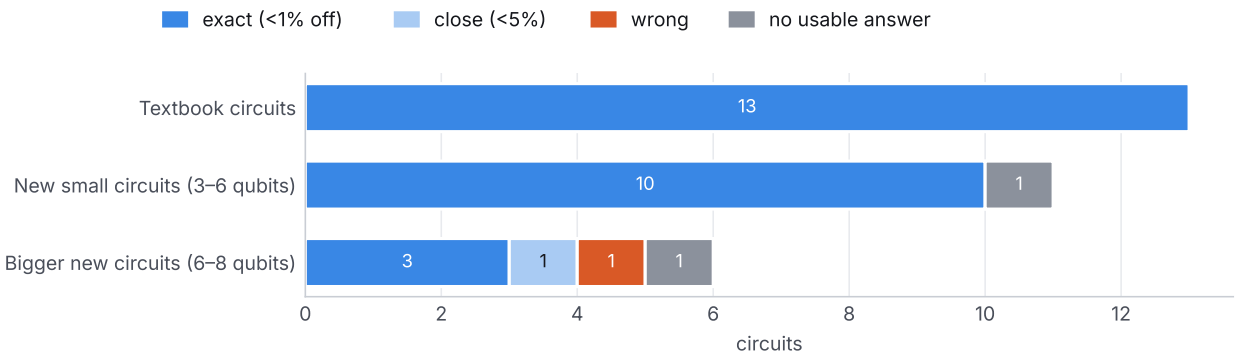


Figure 13 muse-spark-1.3 predicting circuits alone, scored by exact simulation. The model is reliable on what it has seen and on small new circuits, and unreliable where hand calculation grows exponentially harder.

The first run disproved the easy version of the claim: the model was exact on 13 of 13 textbook circuits and on 10 of 11 new small ones, and the page says so. It failed where the calculation gets hard. It gave no usable answer for the 31-gate QAOA circuit. On the bigger circuits it was exact on 3 of 6, close on 1, wrong on 1, and gave no answer on 1. The wrong answer was confident: it described every outcome as equally likely (1/16 each, "a near-maximally mixed marginal") and put 43% of the probability in the wrong place. Nothing in its wording signalled doubt.

CIRCUIT	QUBITS	GATES	MODEL ALONE	PROBABILITY MISPLACED
Bigger circuit 1 (6 qubits, 25 gates)	6	25	no answer	–
Bigger circuit 2 (6 qubits, 25 gates)	6	25	close	1.4%
Bigger circuit 3 (7 qubits, 22 gates)	7	22	matched	0.0%
Bigger circuit 4 (7 qubits, 20 gates)	7	20	wrong	43.2%

CIRCUIT	QUBITS	GATES	MODEL ALONE	PROBABILITY MISPLACED
Bigger circuit 5 (8 qubits, 26 gates)	8	26	matched	0.0%
Bigger circuit 6 (8 qubits, 22 gates)	8	22	matched	0.0%

"Ask the AI alone", under the editor, repeats the test live on any circuit. Offline it refuses rather than inventing an answer. The models table in the app names the check behind every model step.

15.2 The AI referee

If one model can be wrong, can a second model catch it? The referee tests this. Muse Spark answers three questions about a circuit. Muse Wonder agrees with or disputes each answer, giving its own value. Muse Spark then stands by its answer or concedes. Exact simulation grades every position: probabilities within 0.02, the most likely outcome with ties allowed, and entanglement by the negativity of the two-qubit reduced state.

A first version let Muse Spark choose its own claims. It chose safe ones and was right on 3 of 3 every time, which tests nothing. The referee therefore now asks the same three questions of every circuit: the most likely outcome, the probability of the second most likely one (picked from the exact odds; the models are not told why) and whether a given pair of qubits is entangled. The test used the six bigger circuits from the AI-alone benchmark and two smaller new ones, 24 answers in all.

MEASURE	RESULT
Muse Spark's answers right as first stated	17 of 24
Answers right after the debate	19 of 24
Wrong answers Muse Wonder caught with the right value	4 of 7
Right answers Muse Wonder disputed (false alarms)	5
Right answers Muse Spark was talked out of	2

Debate helped on balance: 17 right before, 19 after. But it also added errors of its own. Muse Wonder raised 5 false alarms against right answers and talked Muse Spark out of 2 of them, and it let 3 of the 7 wrong answers through. Twice the two models agreed that a pair of qubits was entangled when its negativity was exactly zero (docs/REFEREE.md). A debate between models cannot tell which challenge was right; only the simulator can. That is why Inkubit checks instead of debating.

15.3 The ML verifier

Exact density-matrix simulation is the referee everywhere in Inkubit, but it costs time that grows exponentially. The ML verifier is a fast first opinion: it predicts how much of a circuit's answer survives realistic noise (the Hellinger fidelity against the ideal distribution, at a noise scale from 0.05× to 5×), with a calibrated error bar.

- **Inputs (16):** structure (qubits, measured qubits, gates by size, depth, two-qubit depth, the busiest qubit), the ideal distribution (entropy, largest outcome, outcomes above 10^{-3}), the noise scale, the rule of thumb (the product of gate and readout success rates), and two physics-informed inputs computed on the ideal distribution alone: exact readout errors, and a global

depolarizing model. Those two were added after the first model was systematically optimistic on textbook circuits. Structure alone cannot see that a Bell pair's outcomes are two bit flips apart, so that every readout error leaves the answer.

- **Data:** 6,000 random circuits of 1 to 7 qubits in five styles (gate soup, layered, few gates, mirror circuits with certain answers, reversible logic), labelled by exact density-matrix simulation (19 ms each on average); 4200 for training, 900 for calibration and 900 held out.
- **Model:** an ensemble of five small tanh networks (16-48-48-1), trained in NumPy with Adam. The ensemble mean is the prediction. The interval is adaptive split conformal: the error divided by the ensemble's spread (plus 0.01), calibrated at the 90th percentile.

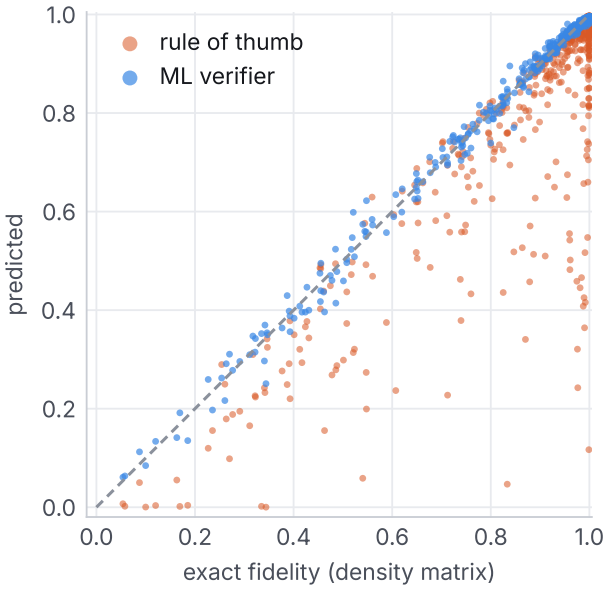


Figure 14 Held-out circuits: predicted against exact fidelity. The ML verifier (blue) hugs the diagonal; the rule of thumb (orange) falls far below it.

	ML VERIFIER	DEPOLARIZING MODEL	RULE OF THUMB
Mean error, 900 held-out circuits	0.0099	0.0127	0.0867
Mean error, 60 textbook cases	0.0201	0.0216	0.0839
90% interval held (held-out)	90%	–	–
90% interval held (textbook)	78%	–	–
Time per prediction	151 µs	–	–

About 128 times faster than the exact label. The coverage guarantee holds for circuits like the training set. That is why every prediction up to 10 qubits is still checked exactly, and why a miss is shown as a note about the model, never as a change to the result.

15.4 A real IBM quantum computer

Everything above is simulation. To test the simulations against reality, `scripts/hardware_check.py` ran four circuits (a Bell pair, GHZ-3, Bernstein-Vazirani and two-qubit Grover) on **ibm_fez**, the least busy device on the IBM Quantum Platform's Open plan (job `dbjkc0pkdqps7390opdg`, 4,096 shots each, 2026-11-01, done 10 seconds after submission). The circuits were transpiled at optimisation level 3, which left one or two two-qubit gates each. The measured counts were compared, by total variation distance, with four predictions:

- the **noiseless** odds;
- Inkubit's **generic realistic preset**, the default noise model of the whole app;
- a **digital twin**: Qiskit Aer with `NoiseModel.from_backend`, built from the device's own calibration data and run on the same transpiled circuit;
- Inkubit's **calibrated preset**, added after this run: its own noise model with the median of a calibration snapshot of the same device (below).

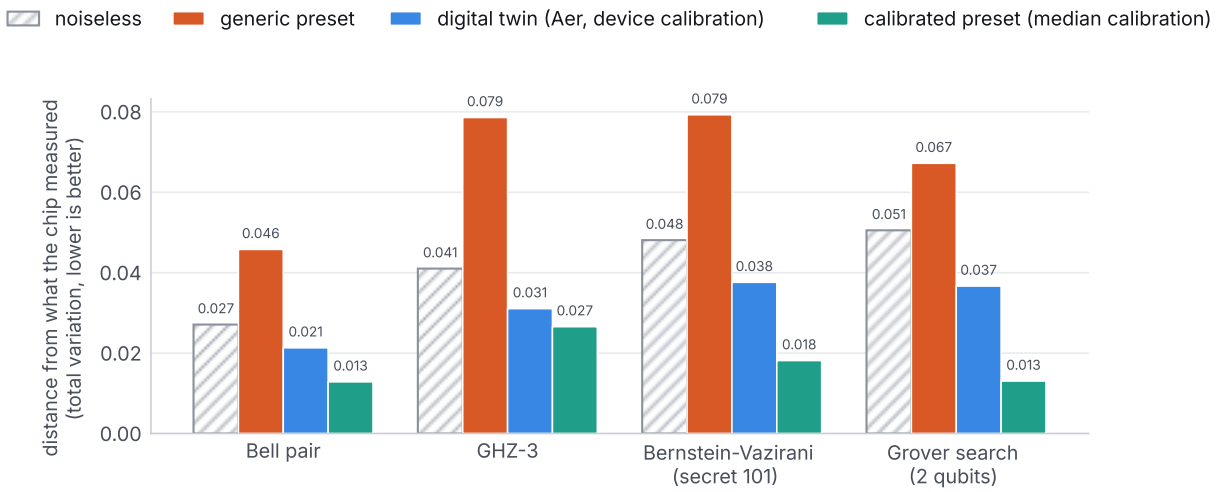


Figure 15 Distance between what ibm_fez measured and each prediction (lower is closer). The twin beat the noiseless odds and the generic preset on 4 of 4 circuits; the calibrated preset was closer than the twin on 4 of 4.

CIRCUIT	2Q GATES	NOISELESS	GENERIC	TWIN	CALIBRATED	SHOT NOISE ALONE
Bell pair	1	0.027	0.046	0.021	**0.013**	0.008
GHZ-3	2	0.041	0.079	0.031	**0.027**	0.009
Bernstein-Vazirani (secret 101)	2	0.048	0.079	0.038	**0.018**	0.004
Grover search (2 qubits)	2	0.051	0.067	0.037	**0.013**	0.003

The digital twin predicted what the chip measured more closely than the noiseless odds and the generic preset on **4 of 4** circuits, at distances of 0.021 to 0.038. The finding that matters more for the app is the second one. The generic preset (0.046 to 0.079) was further off than the noiseless prediction on 4 of 4 circuits: it is calibrated to an older, noisier class of chip, and this one is better.

That finding became a feature. `scripts/calibration_snapshot.py` reads the device's reported calibration and keeps the median over its 156 qubits and couplers: on ibm_fez (Heron r2), a two-qubit error of 0.25%, one-qubit error 2.9×10^{-4}, readout errors of 0.7% and 1.5%, T1 135 µs and T2 101 µs, against the generic preset's 1.5%, 0.1%, 2% and 4%. A reported gate error already includes decoherence during the gate, so the thermal part is taken out before the rest becomes depolarizing noise; a test checks that each gate's composed error equals the reported one. The result is a fourth preset, **Calibrated**, in the run bar and in every analysis. Rescored on the stored counts, it came within **0.013 to 0.027**, closer than the twin on 4 of 4. That is still above the 0.003 to 0.009 that shot noise alone would leave, so part of the chip's behaviour is unmodelled. Two cautions are stated in the app: the snapshot was taken a few hours after the run, and medians describe a typical qubit rather than the best patch a compiler picks.

> **WHAT THIS DOES AND DOES NOT SHOW**
>
> Four small circuits, with one or two two-qubit gates each, on one device on one day. They show that Inkubit's pipeline reaches real hardware, and that a calibrated twin predicts it well at this scale. They are not a benchmark of the device. An offline rehearsal (`--fake` , on a simulated backend) checks the pipeline in CI and is never shown as hardware data. A test refuses to quote hardware numbers from a rehearsal.

15.5 AI-discovered circuits on the chip

The discovery lab's claims (Chapter 7) were made in simulation. `scripts/discovery_hardware.py` reruns the lab's search for three goals and sends each goal's best proven circuit to the chip, together with the baselines it beat: 23 circuits on ibm_fez (job `dbjpr9pffo6c73ba4adg`, 4,096 shots each, 2026-11-01). GHZ-7 runs on seven physical qubits wired like a 7-qubit Falcon chip (118, 109, 108, 110, 98, 111, 112, picked by Qiskit's noise-aware layout for the discovered circuit). Every variant is compiled onto that same patch, so the textbook circuit pays the SWAPs the lab counted: 10 two-qubit gates on the chip against 6.

Computational-basis counts cannot tell a GHZ state from a classical 50/50 mixture of all-0 and all-1, so the GHZ circuits also measure a parity oscillation over eight phases. The fidelity estimate is F = (P(0…0) + P(1…1))/2 + |coherence|, with error bars from shot noise; both are tested against exact states and against repeated sampling. A value above ½ certifies genuine 7-qubit entanglement.

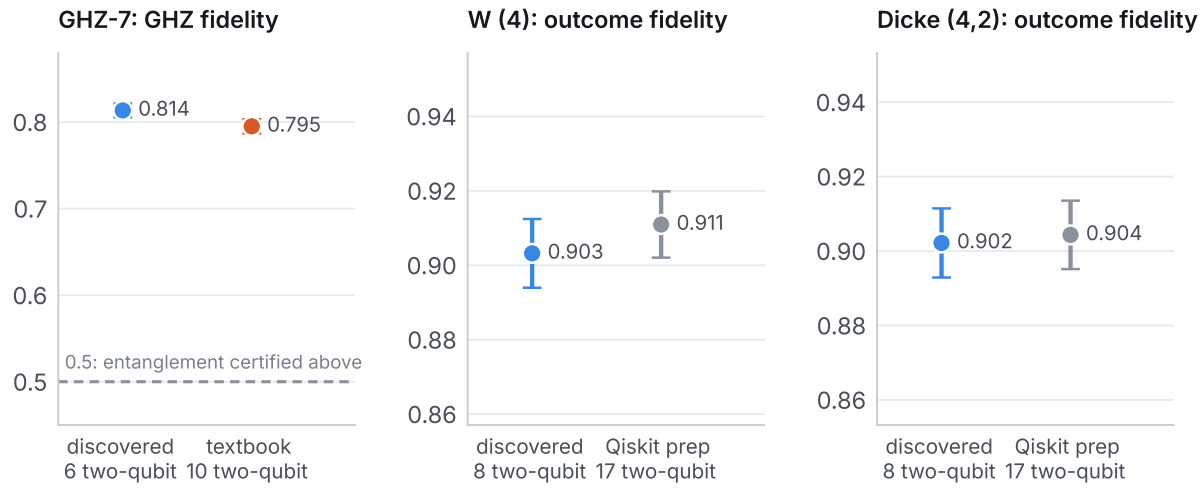


Figure 16 The discovered circuits on ibm_fez against the baselines they beat; bars span two standard errors of shot noise either way, and the axes do not start at zero. Left: GHZ fidelity (above the dashed 0.5, genuine 7-qubit entanglement). Middle and right: outcome fidelity of the computational-basis odds, where the intervals overlap.

GHZ-7: Muse Spark's own layout reached F = 0.814 ± 0.004 (populations 0.861, coherence 0.766) against 0.795 ± 0.004 for the textbook circuit, 3.2 standard errors apart. Both certify genuine 7-qubit entanglement. Qiskit's generic state preparation, 197 two-qubit gates, kept an outcome fidelity of 0.34. **W and Dicke:** compiled freely for the chip, the discovered circuits used 8 two-qubit gates against 17, yet their measured odds tied with Qiskit's within shot noise (-1.2 and -0.3 standard errors). Computational-basis odds cannot see phase errors, which is much of what fewer gates save, and the page reports a tie, not a win. The digital twin was optimistic about every circuit in this run (it predicted 0.903 for the discovered GHZ circuit) but ranked them in the same order.

GOAL	CIRCUIT	CNOTS (LAB)	2Q ON CHIP	GHZ FIDELITY	OUTCOME FIDELITY	TWIN (OUTCOME)
GHZ on all 7 qubits of an IBM heavy-hex chip	Muse Spark's layout #4	6	6	0.814 ± 0.004	0.860 ± 0.005	0.933
	Textbook circuit, compiled onto the wiring	10	10	0.795 ± 0.004	0.841 ± 0.006	0.926
	Qiskit state preparation, optimisation level 3	197	197	–	0.341 ± 0.007	0.623
W state on 4 qubits	Search #1	5	8	–	0.903 ± 0.005	0.958
	Qiskit state preparation, optimisation level 3	11	17	–	0.911 ± 0.004	0.953

GOAL	CIRCUIT	CNOTS (LAB)	2Q ON CHIP	GHZ FIDELITY	OUTCOME FIDELITY	TWIN (OUTCOME)
Dicke state $\|D(4,2)\rangle$	Search #3	5	8	–	0.902 ± 0.005	0.962
	Qiskit state preparation, optimisation level 3	11	17	–	0.904 ± 0.005	0.953

15.6 IBM Quantum: placement and teleportation

The app now maps 3 IBM Heron chips qubit by qubit from their own calibration, and an advisor scores every way to place a circuit on them: each embedding of the circuit's two-qubit gates in the chip's working couplers, ranked by the chance that no gate or readout fails. Two experiments in one job on ibm_fez (`dbk6mf1kdqps7391s3q0`, 63 circuits) tested what the app now claims.

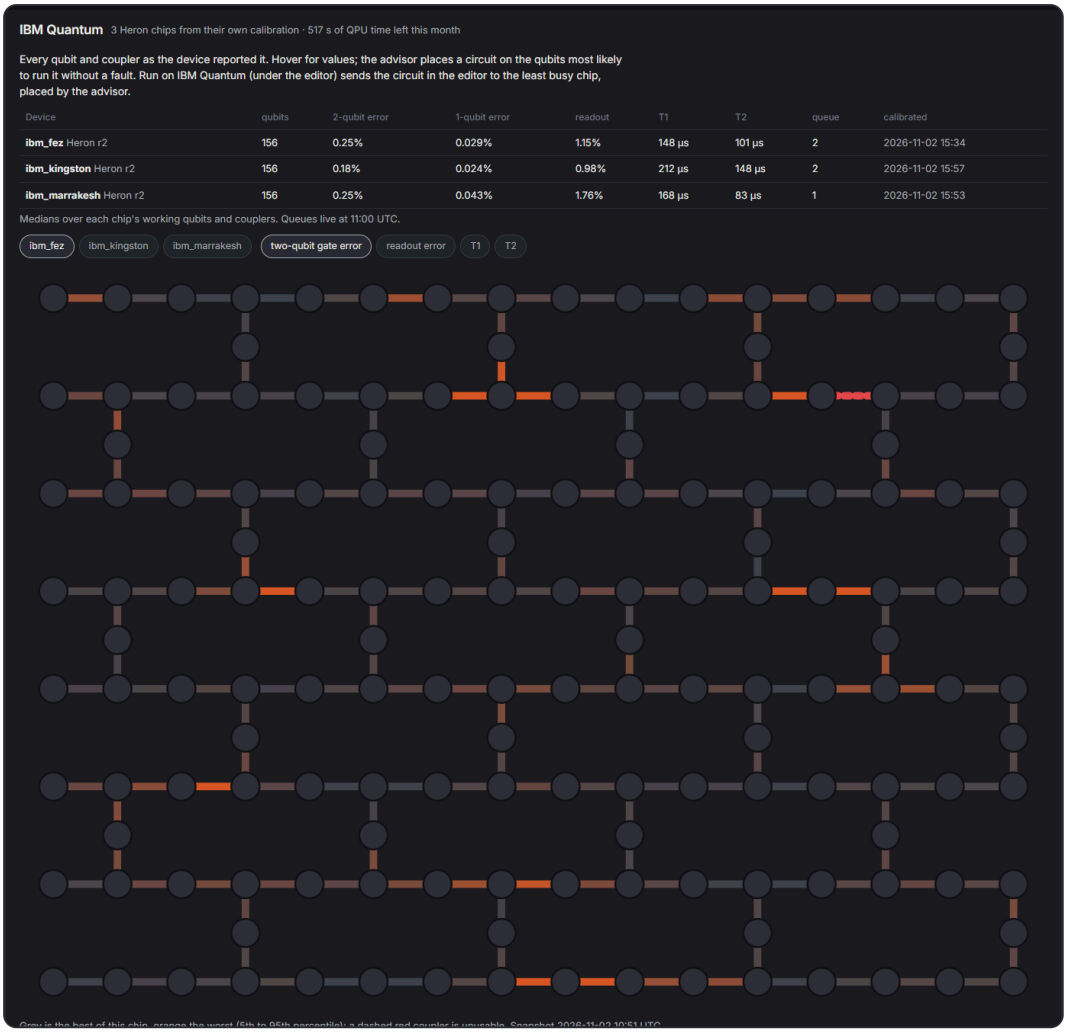


Figure 17 The IBM Quantum panel: three Heron r2 chips, and ibm_fez's 156 qubits and 176 couplers coloured by two-qubit gate error (grey best, orange worst, dashed red unusable).

Placement matters. The advisor scored 804 placements of a 5-qubit GHZ chain from a snapshot taken minutes earlier. The best, median and worst ran on exactly those qubits: GHZ fidelity 0.894 ± 0.003, 0.814 and 0.285, in the advisor's order (its scores: 0.965, 0.899, 0.567). The best and the worst placement were 95 standard errors apart, and on the worst qubits the state no longer certifies entanglement. Same circuit, same chip, same hour; only the qubits differed.

Teleportation with real feed-forward. This run came first, from a script; feed-forward is now part of the app's format and verifier as well (decision D83), and the gallery's feed-forward teleportation sends the same if_test circuit to the chip from the editor. A qubit was teleported with the Bell measurement in the middle of the circuit and X and Z applied by the chip in real time, only when the measured bits said so (qubits 110, 109, 118). The six axis states form a 2-design, so their average fidelity equals the average over all states: **0.924 ± 0.002**, against 2/3 for the best classical scheme, and 0.499 without the corrections, as theory says.

15.7 The proof sheet: every check on every circuit

A run is verified by one comparison (Chapter 5). The proof sheet asks more, of the circuit and of Inkubit itself: 11 independent checks of 9 kinds, each guarding against a different mistake. Two ideal simulators (Qiskit's quantum_info and Aer's C++) and two noisy ones (closed-form channels and Aer's density matrix) must agree. The noisy state must be a physical density matrix. The circuits Qiskit compiles for three chip wirings must be the same unitary, ancillas and routing included: translation validation of the compiler. Exports to OpenQASM 2 and 3 must read back to the same circuit. Sampled shots must pass the shot-noise bound and a G-test. The generated script must verify in the sandbox. Renaming the qubits, or inserting gate pairs that cancel, must leave the answer unchanged (metamorphic tests). It runs on any circuit from the editor in about eight seconds.

Over the whole gallery (28 circuits) it recorded **299 passes and 0 failures**, 1 check skipped with its reason and 8 notes, where the ML verifier's interval missed the exact value, consistent with its coverage on textbook circuits. A test breaks Qiskit's compiler on purpose (dropping one CNOT) and requires the compilation check to fail. Its first run also found a real defect: Inkubit's OpenQASM 2 export used `sx`, `cp` and `swap`, which the standard `qelib1.inc` does not define, so strict readers rejected it. The app's own read-back had used Qiskit's lenient mode and hidden this. The export now defines those gates in the file from standard ones.

CHECK	KIND	GUARDS AGAINST	PASS	FAIL	SKIP	NOTE
The circuit is well formed	Specification	a gate on a qubit that does not exist, a measurement in the middle, a typo in a gate name	28	0	0	0
The generated script runs and verifies	Generated code	a code generator that writes a different circuit than the spec	28	0	0	0
Two ideal simulators agree	Independent engines	a bug in one simulator	28	0	0	0
Two noisy simulators agree	Independent engines	a noise model built differently in two places	27	0	1	0
The noisy state is a physical state	Physics	a noise channel that is not trace-preserving or not completely positive	28	0	0	0
Qiskit's compiled circuits are proven equal	Compilation	a compiler bug, or a routing permutation applied the wrong way round	28	0	0	0
Exports read back to the same circuit	Languages	an export that drops a gate, flips a control or loses an angle	28	0	0	0
Sampled shots match the exact odds	Statistics	a sampler or a readout model that is subtly biased	28	0	0	0
Renaming the qubits changes nothing	Metamorphic tests	a simulator or spec reader that depends on qubit order	28	0	0	0
Gates that cancel change nothing	Metamorphic tests	a gate implemented with the wrong matrix or the wrong convention	28	0	0	0
The ML verifier's interval holds the exact value	Learned screen	nothing about the circuit: it grades the fast model against the exact one	20	0	0	8

15.8 The simulation observatory

Five exact simulations show where quantum technology stands, each with its key number written into one sentence from the data. Satellites versus fiber is in Chapter 12. The other four:

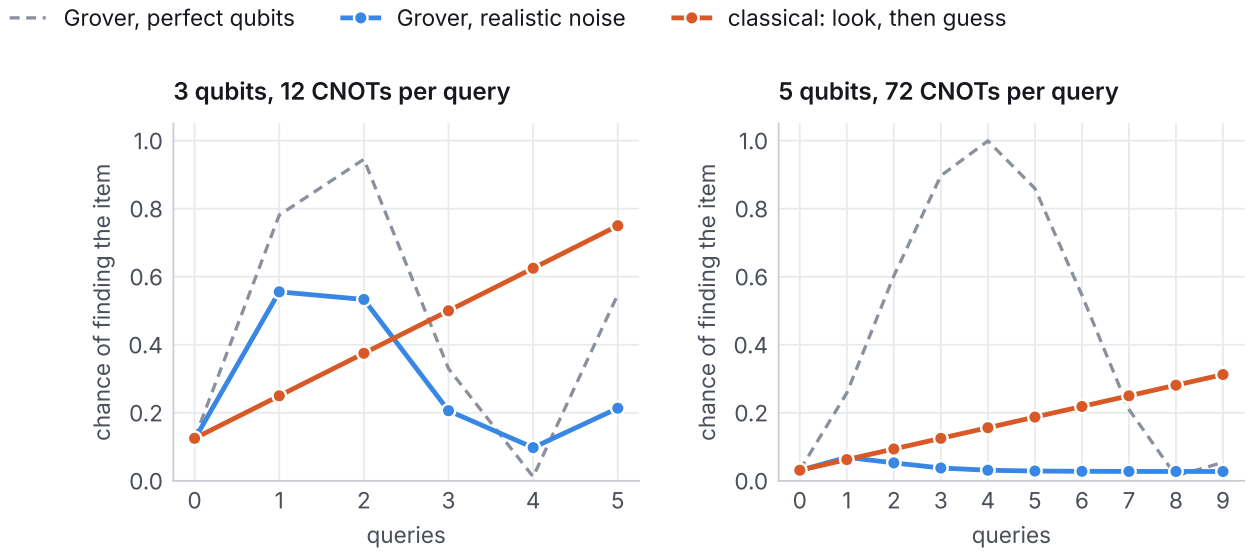


Figure 18 Ideal Grover finds the 1 marked item among 32 with probability 100% after 4 queries. On today's hardware (72 CNOTs per query) it peaks at 7% after 1, no better than a classical search with the same looks: noise has erased the speedup.

Grover versus noise. Two to five qubits, one marked item, transpiled to native gates and evolved as exact density matrices with the verifier's gate channels. With perfect qubits, the search finds the item with near certainty. With today's noise the advantage shrinks as the circuits grow, and at five qubits the speedup is gone.

Entanglement growth. Random two-qubit gates entangle the two halves of 10 qubits layer by layer; after 18 layers they hold 4.16 bits, within 3% of Page's limit for a random state (4.28 bits). From then on there is no compact description left for a classical computer to exploit. **The quantum walk.** After 60 steps the quantum walker has spread 32.5 sites against 7.7 for a coin-tossing walker, 4.2 times as far, and the gap keeps growing: interference turns diffusion into a front that moves at constant speed. **The exponential wall.** Every extra qubit doubles the memory: a 16 GB laptop holds at most 29 qubits exactly, and 49 qubits already need 9 PB, the scale of the largest supercomputers. 60 qubits would need 18 EB.

CHAPTER 16

Engineering

Tests that run offline in minutes, browser tests that drive the built app, a test that holds the project's own documents to its data, a sandbox and a public mode for a shared host, and a deployment that fits a free container.

16.1 Tests

The backend has **591 tests**. All of them run offline, against the mock models and with no key, in about six minutes. They cover:

- specification validation, including readable errors with paths, and code generation with its static safety check;
- the verifier's statistics, and the repair loop with every planted fault, including the honest-failure path;
- exact insights checked against Qiskit Aer (to 1e-8), Wootters concurrence against Qiskit, CHSH against brute force;
- the optimiser's rewrite rules, fuzzed on random circuits; the challenge reference solutions reaching exactly par;
- network physics and its sanity checks; orbits against Skyfield; solar position at the solstice; the J2 nodal drift;
- chemistry against Szabo and Ostlund's tabulated values; QAOA and BB84 against Qiskit on the generated circuits;
- the decoders' layouts, the exact matcher against PyMatching, and the stored precomputed results against a fresh computation;
- the live adapters against recorded request shapes from Meta's documentation, and the API end to end.

`pytest -m live` makes one cheap real call per Meta model. It is skipped without a key, and cached after the first run.

16.2 Our own numbers, held to the app's rule

The app traces every number a model writes to the tool that computed it. The project applies the same rule to itself. `tests/test_claims.py` rebuilds every number that the README and the submission text quote from the data file it comes from: the hardware run, the referee, the learning curves, the codes, the whole paper, both decoders, the mission benchmark, the AI-alone benchmark, the ML verifier and the RSA-2048 comparison. It fails if a document quotes anything else, and it refuses a hardware number that comes from a rehearsal rather than a real device.

`scripts/check_submission.py` checks what the hackathon form needs: a public repository, video and demo links, the video's length, the thumbnail, test counts, quoted claims, no secrets in history, CI on the latest commit, and the deadline. Its first run caught the documents quoting 448 tests after the suite had grown to 468. This report is built the same way: `scripts/build_report.py` reads every benchmark number from data, and an unknown placeholder stops the build.

16.3 Continuous integration and browser tests

GitHub Actions runs three jobs on every push: the offline backend suite, the frontend type-check and build, and the end-to-end browser tests. The 6 browser tests start the server on the offline models and drive the built app in Chromium. They check that every section shows its panels; that a sample sketch goes all the way to Verified; that the guided tour lands on every step and leaves a finished mission and discovery run; that the satellite planner flies a real satellite; and that no section scrolls sideways on a 390-pixel phone screen. The section list and tour steps are read from the app's own source, so the tests follow the app as it changes.

They caught a real bug. On a cold server, the tour scrolled to the decoder before the pulse lab above it had loaded. The pulse lab then grew and pushed the decoder 878 px down the page. The fix holds a panel in place for up to 15 s while the layout above it settles, and releases it the moment the visitor scrolls, clicks or types.

16.4 Security and public mode

- **Model-written code** runs only after an AST check, in a separate Python process with a timeout and an environment scrubbed of keys, tokens and secrets. In public mode it is never executed at all: the agent runs Inkubit's own template code, and the planted-fault demo still works because the faults go into that code.
- **Costs on a shared host** are capped at 300 billed model calls per UTC day, counted only on cache misses. After the cap, the labelled offline models answer.
- **Secrets.** Before the repository was made ready to publish, every commit then in its history (78) was searched for key patterns (Meta, Hugging Face, GitHub, AWS, private keys) and for the literal Meta key. There were no hits in history, tracked files or the built frontend; `.env` was never committed. The deploy script never uploads `.env`.
- **A fresh clone**, following the README exactly with a new virtual environment and no `.env`, starts in offline mode, every feature endpoint answers, and the test suite passes. The only skipped tests need the dev extras, Skyfield and Cirq.

16.5 Hosting on a free container

The Dockerfile builds the frontend with Node and serves everything from Python 3.13, as a non-root user, on port 7860. Hugging Face now requires a PRO account for Docker Spaces, so the free route is a container host's free tier, with 512 MB of memory and a tenth of a CPU, asleep when idle. Memory was measured in public mode:

MOMENT	MEMORY IN USE
After start	206 MB
After the labs and a mission	223 MB
During a discovery run	255 MB
Server plus sandbox at the peak of a sketch run	385 MB

A tenth of a CPU makes heavy actions about ten times slower, and the start-up warm-up would have spent minutes of it after every wake. The input-free results (the observatory, the pulse sweep and its DRAG calibrations, 37 KB) are therefore stored with the app, and a test checks that they equal a fresh computation. Koyeb is preferred: it wakes in seconds and sleeps after an hour. A deploy link in the README fills in the repository, the Dockerfile, the port and the free instance.

16.6 The demo video, recorded by a script

`scripts/record_demo.py` drives the real app in headless Edge and records the browser's own screencast at 1920 × 1080 (H.264, via the ffmpeg bundled with imageio-ffmpeg), with caption cards and a visible cursor laid over the page. Its rules keep it honest. Every number in a caption is read from the recorded results, never typed in. Captions name a Meta model only when the server reports live Meta mode. Waits for the sandbox or for model calls are sped up 4× only with an on-screen tag. The current take runs 2:56, opens on the 3D scene, shows the feed-forward error-correction cycle and was recorded against the live models. A timed voice-over script sits beside it. The README's opening GIF and the video thumbnail are cut from the same recording.

CHAPTER 17

Limitations and next steps

What Inkubit does not do, stated as plainly as what it does. Each limitation is also stated in the app or the README where it applies.

17.1 Limitations

- **Verification sees statistics.** It cannot see a pure phase error that leaves the measured distribution unchanged, and it does not catch changes smaller than its threshold, such as an angle off by a few degrees. The optimiser's proofs, tomography and the Bell tests look further, but they are not run on every result.
- **Circuits.** Feed-forward (mid-circuit measurement, qubit reuse, reset and conditions on several measured bits) runs, verifies exactly and goes to IBM chips, but each classical bit is written once and the tools built on a single state (step-through, live state, optimizer, proof sheet) refuse it. Circuits are capped at 12 qubits, 10 counting the qubits the exact model adds, and the noisy analyses at 10.
- **The sandbox** is a guard rail, not an operating-system sandbox; on a shared host model code is not run.
- **The network model** is analytic and simplified: simple BBPSSW purification, no multi-pair emission and lumped atmospheric loss. Its numbers are order-of-magnitude estimates, not engineering predictions. For the real satellites, pass times, elevations and clouds are real data, but key rates are estimates.
- **Noise.** The decoders use phenomenological noise, not circuit-level noise. The generic noise preset is too pessimistic for a modern IBM chip; the calibrated preset uses median errors, not those of the patch a compiler picks. The H_2 lab's mitigation assumes the calibrated noise map is known.
- **Learned components** carry their guarantees only for data like their training data: the ML verifier's interval held for 78% of textbook cases against 90% on held-out ones. That is why exact checks still run.
- **Discovery and codes.** "Fewest CNOTs found" is not a proof of minimality. The new biased-noise code's distances are proven, but its performance under a decoder has not been simulated.
- **Offline mode** recognises only the three bundled sketches, and explains in English only.

- **Untested paths.** The Muse Code CLI repair backend is opt-in and untested here, because the CLI was not installed. Meta's open-weight Glimmer model runs, but too slowly to drive the app, and no results from it are claimed.

17.2 Next steps

1. **Calibrate per patch.** The median-calibrated preset already predicts the chip better than the generic one and than a digital twin did; per-qubit errors for the patch the compiler picks are the next step.
2. **Circuit-level noise for the decoders**, so that Λ can be compared with hardware like for like, and a decoder study of the AI-designed biased-noise code.
3. **Repeated error-correction cycles**: the one-cycle bit-flip code with a reused helper qubit exists; several cycles need more than the 10 qubits the exact model allows today.
4. **Full-state tests on hardware:** the discovered W and Dicke circuits tied in the computational basis; tomography would show what their fewer gates save in phase coherence.
5. **Multi-page lab notebooks:** sketches that span pages, read as one experiment.

CHAPTER 18

Built during the hackathon

Everything in the repository, in the video and in this report was built during the building period, 27 October to 4 November 2026, by one developer. Nothing was carried over from an earlier project.

18.1 The record

The git history is the record. The first commit, which defines the specification format, is from 27 October 2026 at 14:57 ET. The history so far has 100 commits over 7 days. Third-party code is limited to open-source libraries installed from PyPI and npm and listed in the README. The images were generated with Muse Image or drawn by the repository's own scripts.

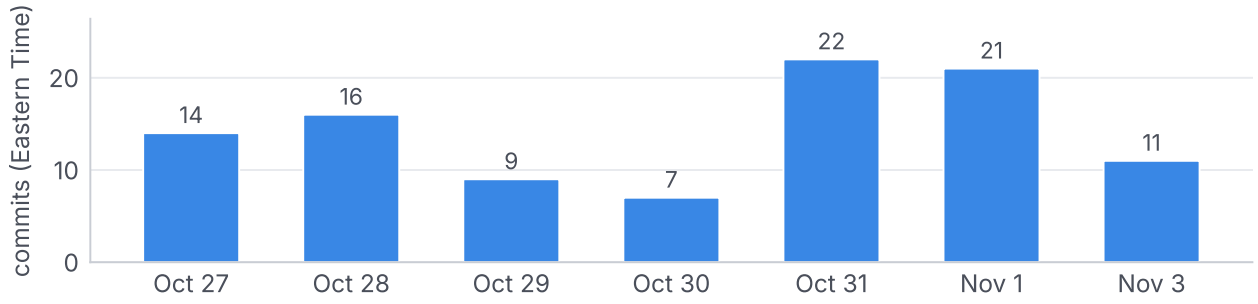


Figure 19 Commits per day, Eastern Time, from the git log at build time.

18.2 Day by day

DAY (ET)	WHAT WAS BUILT
Oct 27	Specification format, code generation, noise-aware verification, the self-repair agent, sketch reading with SAM 3.1 and Muse Spark, the API, the React app, explanations and diagrams, the quantum network model
Oct 28	Algorithm gallery, gate-by-gate inspector and circuit analyses, entanglement purification, describe in words, ask about a run, chip routing, 12 languages, HTML reports, the planted-fault benchmark
Oct 29	Challenge mode, 3D Bloch spheres, the optimiser with exact proofs, zero-noise extrapolation, Bell tests, tomography, the error-correction lab, voice input
Oct 30	First live run against the Meta Model API, Shor's algorithm, Muse Voice Transcribe, the QAOA party lab, the bug hunt
Oct 31	AI alone vs Inkubit, the molecule lab, the BB84 spy lab, use cases with Muse Image, Ask Inkubit (tool calling) with a Muse Wonder deep mode, the first demo video
Nov 1	The Quantum Rosetta; the simulation observatory; mission control; the ML verifier; seven more algorithms; the discovery lab, distributed computer, resource estimator, live board, satellite planner, paper checker, ML decoder and pulse lab. Then real satellites on SGP4, the distance-5 decoder, browser tests in CI, the claims test and submission checker, the free-container fit, the four new labs (learning under proof, whole papers, AI-designed codes, the referee), the run on IBM's ibm_fez, the 2:48 video, the discovered circuits on the chip and the calibrated preset

18.3 How the work was done

Every design decision was written down with its reason as it was made (85 of them, Appendix B), including those that went against an easy claim: when the first AI-alone run showed the model exact on small circuits, the page said so and the test moved to bigger ones. Every feature that uses a model got an exact check, every quoted result came from a script that can be run again (Appendix C), and bugs found by tests are recorded with their fixes, because they show where a system like this can go wrong without anyone noticing.

APPENDIX A

API reference

The server exposes 74 endpoints, grouped here by area. Long-running work (jobs, missions, discovery and learning runs) returns an id at once, and its `/events` endpoint streams every step as server-sent events.

AREA	ENDPOINTS
health	`GET /api/health`
headline	`GET /api/headline`
examples	`GET /api/examples` · `GET /api/examples/images/{name}`
spec	`POST /api/spec/validate`
proofsheet	`GET /api/proofsheet/benchmark` · `POST /api/proofsheet`
insights	`POST /api/insights`
state	`POST /api/state`
optimize	`POST /api/optimize`
missions	`POST /api/missions` · `GET /api/missions/{mission_id}` · `GET /api/missions/{mission_id}/report` · `GET /api/missions/{mission_id}/events`
mlverify	`POST /api/mlverify` · `GET /api/mlverify/report`
discovery	`GET /api/discovery/goals` · `POST /api/discovery` · `GET /api/discovery/{discovery_id}/events` · `GET /api/discovery/{discovery_id}`
learning	`GET /api/learning/benchmark` · `POST /api/learning` · `GET /api/learning/{learning_id}/events`
referee	`GET /api/referee/benchmark` · `POST /api/referee` · `GET /api/referee/{referee_id}`
paper	`GET /api/paper/examples` · `POST /api/paper` · `GET /api/paper/{paper_id}`
codes	`GET /api/codes` · `POST /api/codes/analyse`
distributed	`POST /api/distributed`
estimate	`GET /api/estimate/presets` · `POST /api/estimate`
live	`POST /api/live/read`
orbits	`GET /api/orbits/cities` · `GET /api/orbits`
reproduce	`POST /api/reproduce`
decoder	`GET /api/decoder`
pulse	`GET /api/pulse`
hardware	`GET /api/hardware` · `GET /api/hardware/discovery`
ibm	`GET /api/ibm/status` · `GET /api/ibm/device/{name}` · `POST /api/ibm/advise` · `POST /api/ibm/run` · `GET /api/ibm/run/{job_id}` · `GET /api/ibm/experiments`
observatory	`GET /api/observatory`
lab	`GET /api/lab/repetition` · `GET /api/lab/shor` · `GET /api/lab/chem` · `GET /api/lab/qkd` · `POST /api/lab/qaoa`
copilot	`POST /api/copilot`
rosetta	`POST /api/rosetta` · `GET /api/rosetta/dictionary` · `POST /api/rosetta/photo`
arena	`POST /api/arena` · `GET /api/arena/benchmark`
bughunt	`GET /api/bughunt/round` · `POST /api/bughunt/guess` · `POST /api/bughunt/muse`
challenges	`GET /api/challenges` · `POST /api/challenges/{challenge_id}/check` · `POST /api/challenges/{challenge_id}/hint`

AREA	ENDPOINTS
jobs	`POST /api/jobs/image` · `GET /api/jobs/{job_id}/report` · `POST /api/jobs/{job_id}/ask` · `POST /api/jobs/text` · `POST /api/jobs/spec` · `GET /api/jobs/{job_id}` · `GET /api/jobs/{job_id}/events`
transcribe	`POST /api/transcribe`

All endpoints are served by `backend/inkubit/api/app.py`. The built frontend is served from the same origin. `/api/health` reports the model provider (Meta live, mock, or mock fallback), and the recorder and the interface read it before naming a model.

APPENDIX B

Decision index

Every design decision is recorded in `DECISIONS.md` with its reason, newest last. This index lists all 85; the chapters above cite the ones that matter most.

	DECISION
D1	Project location and identity (2026-10-28)
D2	Meta API docs were fetched, not invented
D3	Which Meta model does which job
D4	Spec design
D5	Python 3.13 venv
D6	Generated code runs in a guarded subprocess
D7	Noise presets
D8	Verification method and thresholds
D9	Agent loop design
D10	Muse Code CLI backend is optional and untested here
D11	Sketch understanding pipeline
D12	Network model assumptions (P9)
D13	Bottleneck search
D14	When SAM finds nothing on a real sketch
D15	Result timing
D16	Automated demo video
D17	New gates and the algorithm gallery
D18	Circuit insights
D19	Entanglement purification
D20	Describe it in words, camera, share links
D21	Ask about a result
D22	Chip layouts
D23	Explanations in 12 languages
D24	Challenge mode
D25	Click-to-compose and the live state

	DECISION
D44	Muse Wonder as the assistant's deep mode
D45	Hackathon rules
D46	The Quantum Rosetta
D47	The simulation observatory, and diagram-style card art
D48	Mission control: plan, run in parallel, repair, verify, trace
D49	The ML verifier
D50	Seven more algorithms
D51	Discovery lab
D52	Distributed quantum computer
D53	When does quantum win?
D54	Live board
D55	Satellite planner
D56	Reproduce a paper
D57	ML error-correction decoder
D58	Pulse lab
D59	Focus pass: one hero, five sections
D60	Guided tour and CI
D61	Public mode and the hosted copy
D62	Mission benchmark and the hardware check
D63	Real satellites instead of a modelled one
D64	Fresh-clone check and a history scan before going public
D65	End-to-end browser tests in CI, and the bug they found
D66	Distance 5: error suppression, and a network that fixes what matching gets wrong
D67	Meta's open-weight models, tried on this laptop
D68	A new video of under three minutes
D69	Presentation, and our own numbers held to the app's rule

	DECISION
D26	What-if slider and network map
D27	Optimizer: proposals are proven, never trusted
D28	Zero-noise extrapolation by global folding
D29	Entanglement web
D30	Bell tests
D31	State tomography
D32	Error-correction lab
D33	Say it
D34	First live run
D35	Say it with Muse Voice Transcribe
D36	Split the party (QAOA Max-Cut)
D37	Bug hunt
D38	AI alone vs Inkubit, measured instead of claimed
D39	Molecule lab (H2 from first principles)
D40	Catch the spy (BB84)
D41	Who it is for
D42	Ask Inkubit: tools, not answers
D43	Images from Muse Image

	DECISION
D70	Hosting on a free container
D71	An AI scientist that learns under proof
D72	Reproducing a whole paper
D73	Code lab: AI-designed error-correcting codes
D74	The AI referee
D75	A real IBM quantum computer
D76	AI-discovered circuits on a real quantum computer
D77	A calibrated noise preset from a real chip
D78	Eight more algorithms, in categories
D79	The proof sheet
D80	IBM Quantum in the app: chip maps, a placement advisor, runs from the editor
D81	Placement and teleportation on a real chip
D82	Renamed to Inkubit, and led with proof and real hardware (2026-11-03)
D83	Feed-forward in the format, the verifier, the editor and IBM runs (2026-11-03)
D84	Reuse, reset, conditions on several bits, and descriptions that ask for feed-forward (2026-11-03)
D85	A second pass against the generated look (2026-11-03)

APPENDIX C

Reproducing every result

Every result in this report comes from a script in the repository. The offline ones run anywhere; the live ones need a Meta key, or an IBM Quantum key for the hardware run, and their answers are cached so a re-run is quick.

C.1 Set up and test

```
py -3.13 -m venv .venv
.venv/Scripts/python -m pip install -r backend/requirements.txt
cd frontend && npm ci && npm run build && cd ..
cd backend && ../.venv/Scripts/python -m uvicorn inkubit.api.app:app --port 8000   #
http://127.0.0.1:8000
cd backend && ../.venv/Scripts/python -m pytest -q                                  # 591 offline
tests
.venv/Scripts/python -m pytest e2e                                                  # browser
tests (after the build)
```

Put `META_API_KEY` and `INKUBIT_MODEL_PROVIDER=meta` in `.env` for the live models; without them everything runs on the labelled offline models.

C.2 Results and the scripts that produce them

RESULT	COMMAND	NEEDS
Planted-fault benchmark (55 of 55)	`scripts/benchmark.py`	offline
Mission control benchmark (30 missions)	`scripts/mission_bench.py`	Meta key
AI alone vs Inkubit	`scripts/arena_bench.py` (`--glimmer` for the open-weight model)	Meta key
Learning under proof	`scripts/learning_bench.py`	Meta key
AI-designed codes	`scripts/codes_bench.py`	Meta key
The AI referee	`scripts/referee_bench.py`	Meta key
Distance-3 decoder	`scripts/train_decoder.py [shots per level]`	offline
Distance-5 decoder and Λ	`scripts/train_decoder_d5.py [million shots]` (`--eval-only` re-scores)	offline
ML verifier	`scripts/train_ml_verifier.py [samples]`	offline
Observatory and pulse sweep	`scripts/precompute.py`	offline
Real hardware run	`scripts/hardware_check.py` (`--fake` rehearses, `--rescore` adds the calibrated preset)	IBM Quantum key
Discovered circuits on hardware	`scripts/discovery_hardware.py` (`--fake` rehearses, `--errors` recomputes error bars)	IBM Quantum key
New algorithms, checked	`scripts/make_algorithms.py`	offline
Proof sheet over the gallery	`scripts/proofsheet_bench.py`	offline
IBM chip snapshots (the map and the advisor)	`scripts/device_snapshot.py`	IBM Quantum key
Placement and teleportation on IBM	`scripts/ibm_experiments.py` (`--fake` rehearses)	IBM Quantum key
Calibrated preset	`scripts/calibration_snapshot.py [device]`	IBM Quantum key
Demo video	`scripts/record_demo.py --out docs/video/…mp4`	running server
Submission check	`scripts/check_submission.py`	offline, `gh` for CI status
This report	`scripts/build_report.py`	system Edge; web fonts if online

The whole-paper result is produced from the app (`POST /api/paper` with an arXiv id) and stored in `examples/paper_examples.json`. The discovery-lab results are reproduced from its panel.

APPENDIX D

Glossary

Bell pair
Two qubits in the state (|00⟩ + |11⟩)/√2: measured, they always agree, yet each alone is a fair coin.

BB84, BBM92
Quantum key distribution protocols: BB84 sends single photons in random bases; BBM92 measures halves of entangled pairs. An eavesdropper raises the error rate (QBER).

Bloch sphere
A picture of one qubit's state as an arrow; an arrow shorter than the radius means the qubit is entangled or noisy.

CHSH, Mermin values
Bell-test numbers: above 2, no classical (local hidden-variable) explanation can produce the correlations.

Concurrence, negativity
Measures of entanglement between two qubits; zero means the pair is not entangled.

Conformal prediction
A way to put an error bar on a model's prediction that holds a stated fraction of the time for data like the calibration set.

CSS code
A stabilizer code whose generators are all-X or all-Z, so bit and phase errors are corrected separately.

Density matrix
The full description of a possibly noisy quantum state; simulating it exactly is what Inkubit's verifier does.

Digital twin
A simulation of a specific device, built from its own calibration data.

Distance (d, dX, dZ)
The fewest single-qubit errors (of any kind, X only, or Z only) that defeat a code without being detected.

DRAG
Derivative Removal by Adiabatic Gate: a pulse shape that cancels leakage and phase errors in weakly anharmonic qubits.

GHZ, W and Dicke states
Families of multi-qubit entangled states; GHZ is all-or-nothing, W shares one excitation, Dicke shares several.

Heavy-hex
IBM's chip wiring, in which each qubit has at most three neighbours.

Hellinger fidelity, total variation distance (TVD)
Ways to compare two probability distributions; TVD is the share of probability in the wrong place.

Λ (Lambda)
The factor by which logical errors fall when a surface code grows by two in distance; above 1 means the bigger code helps.

Leakage
Population escaping from the qubit's two levels into a third level of a transmon.

McNemar test
A paired test that compares two methods on the same samples by counting only where they disagree.

Minimum-weight perfect matching (MWPM)
The standard surface-code decoder: it pairs detection events along the most likely error chains.

Page limit
The average entanglement of a random state between two halves; random circuits approach it.

Phenomenological noise
A decoder-testing noise model with errors on data qubits and on check outcomes each round, but not from individual gates.

Process fidelity
How close two operations are for every input: $|\mathrm{Tr}(U^\dagger V)|^2/d^2$.

QAOA
The quantum approximate optimisation algorithm: alternating cost and mixing layers with tuned angles.

Richardson extrapolation, ZNE
Running a circuit at amplified noise and extrapolating back to zero noise.

SGP4, TLE
The standard propagator for satellite orbits, and the two-line element sets it reads.

Stabilizer code
A quantum code defined by commuting Pauli generators whose measurements reveal errors.

Surface code
A stabilizer code on a 2D grid with only nearest-neighbour checks; the leading design for large machines.

T1, T2
A qubit's energy-relaxation and dephasing times.

Tomography
Rebuilding a state from measurements in many bases.

Transpile
Compile a circuit onto a chip's native gates and wiring, adding SWAPs where qubits are not neighbours.

VQE
The variational quantum eigensolver: a quantum circuit with tuned angles whose measured energy is minimised.

Werner state
A Bell pair mixed with white noise, described by one fidelity F; the standard model of a noisy link.

APPENDIX E

References

1. Beverland, M. E. et al. Assessing requirements to scale to practical quantum advantage. arXiv:2211.07629 (2022).
2. Gidney, C. and Ekerå, M. How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits. *Quantum* 5, 433 (2021).
3. Google Quantum AI and collaborators. Quantum error correction below the surface code threshold. *Nature* 638, 920 (2025).
4. Bausch, J. et al. Learning high-accuracy error decoding for quantum processors. *Nature* 635, 834 (2024).
5. Higgott, O. PyMatching: a Python package for decoding quantum codes with minimum-weight perfect matching. *ACM Transactions on Quantum Computing* 3, 16 (2022).
6. McNemar, Q. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika* 12, 153 (1947).
7. Motzoi, F., Gambetta, J. M., Rebentrost, P. and Wilhelm, F. K. Simple pulses for elimination of leakage in weakly nonlinear qubits. *Physical Review Letters* 103, 110501 (2009).
8. Eisert, J., Jacobs, K., Papadopoulos, P. and Plenio, M. B. Optimal local implementation of nonlocal quantum gates. *Physical Review A* 62, 052317 (2000).
9. Ross, N. J. and Selinger, P. Optimal ancilla-free Clifford+T approximation of z-rotations. *Quantum Information and Computation* 16, 901 (2016).
10. Bennett, C. H. and Brassard, G. Quantum cryptography: public key distribution and coin tossing. Proceedings of IEEE ICCSSP, 175 (1984).
11. Bennett, C. H., Brassard, G. and Mermin, N. D. Quantum cryptography without Bell's theorem. *Physical Review Letters* 68, 557 (1992).
12. Bennett, C. H. et al. Purification of noisy entanglement and faithful teleportation via noisy channels. *Physical Review Letters* 76, 722 (1996).
13. Yin, J. et al. Satellite-based entanglement distribution over 1200 kilometers. *Science* 356, 1140 (2017).
14. Vallado, D. A., Crawford, P., Hujsak, R. and Kelso, T. S. Revisiting Spacetrack Report #3. AIAA 2006-6753 (2006).
15. Horodecki, R., Horodecki, P. and Horodecki, M. Violating Bell inequality by mixed spin-½ states: necessary and sufficient condition. *Physics Letters A* 200, 340 (1995).
16. Mermin, N. D. Extreme quantum entanglement in a superposition of macroscopically distinct states. *Physical Review Letters* 65, 1838 (1990).
17. Wootters, W. K. Entanglement of formation of an arbitrary state of two qubits. *Physical Review Letters* 80, 2245 (1998).
18. Smolin, J. A., Gambetta, J. M. and Smith, G. Efficient method for computing the maximum-likelihood quantum state from measurements with additive Gaussian noise. *Physical Review Letters* 108, 070502 (2012).
19. Temme, K., Bravyi, S. and Gambetta, J. M. Error mitigation for short-depth quantum circuits. *Physical Review Letters* 119, 180509 (2017).
20. Zhou, L., Wang, S.-T., Choi, S., Pichler, H. and Lukin, M. D. Quantum approximate optimization algorithm: performance, mechanism, and implementation on near-term devices. *Physical Review X* 10, 021067 (2020).
21. Page, D. N. Average entropy of a subsystem. *Physical Review Letters* 71, 1291 (1993).
22. Szabo, A. and Ostlund, N. S. *Modern Quantum Chemistry*. Dover (1996).
23. Quantum Algorithm Implementations for Beginners. arXiv:1804.03719 (the paper used in Chapter 9).

24. Meta. Meta Model API documentation (models, chat completions, responses, images, speech-to-text, structured output, tool calling), as listed in `docs/meta-api/SOURCES.md`.